

Danmarks
Tekniske
Universitet



Videnskabelig maskinlæring til simulering af plasmaranddynamikker

AUTHOR

Andreas Bjarnastein Antoft - s224041

June 21, 2026

Contents

1	Introduktion	1
1.1	Baggrund og motivation	1
1.2	Formål og forskningsspørgsmål	2
2	Metode	3
2.1	Approximation af PDE'er med neurale netværk	3
2.1.1	Hårde begrænsninger for randbetingelser	3
2.1.2	Fysik informerede neurale netværk og bløde begrænsninger	4
2.1.3	Valg af rand begrænsning	5
2.2	Fysikken bag HESEL modellen	6
2.2.1	Braginskii's fluidligninger	6
2.2.2	HESEL modellen	10
2.3	Numeriske metoder	14
2.3.1	Rumlig diskretisering	14
2.3.2	Numeriske differentialoperatorer	14
2.3.3	Tidsintegration	15
2.4	Tilgang til eksperiment	15
3	Implementering	17
3.1	Overordnet struktur	17
3.2	Datahåndtering	17
3.3	Modelimplimentering	18
4	Data og eksperiment	19
4.1	Opsætning og generering af data	19
4.1.1	BOUT opsætning	19
4.1.2	Data generering	19
4.1.3	Data visualisering	21
4.1.4	Opdeling af data til eksperiment	22
4.1.5	Domæne for modelarkitektur	23
5	Resultater og diskution	24
5.1	Udforskning af modelarkitektur og hyperparametre	24
5.1.1	Model arkitektur for endelige eksperiment	27
5.2	Endelige eksperiment	28
5.2.1	Endelige eksperiment for PINN_z	28
5.2.2	Endelige eksperiment for PINN	32
6	Konklusion	35
	Figurliste	I
	Tabelliste	II

Nomenklatur	III
Referencer	IV
7 Appendix A	V
8 Appendix B	XI
8.1 Anvendelse af Boltzmann ligning	XI

1 Introduktion

1.1 Baggrund og motivation

Verdens energiforbrug forventes at stige markant i de kommende årtier, samtidig med at behovet for stabile energikilder med lave CO₂ udledninger bliver stadig mere presserende. I den sammenhæng fremstår fusionsenergi som en attraktiv mulighed, fordi den i princippet kan levere store mængder energi med begrænset brændselsforbrug og uden de samme langsigtede affaldsproblemer som konventionel fission [1]. Vejen fra fysisk mulighed til teknologisk realisering er imidlertid lang, blandt andet fordi plasmaet i en fusionsreaktor er et meget komplekst system, hvor små ændringer i lokale forhold kan føre til markante forskelle i transport, stabilitet og energitab.

I denne rapport er fokus rettet mod tokamakken, hvor et varmt, ioniseret plasma holdes på plads af stærke magnetfelter. Særligt interessant er plasmaets randregion, altså overgangen mellem det godt indesluttede plasma og scrape off layer (SOL), hvor feltlinjerne ikke længere er lukkede. Det er netop i denne randzone, at turbulens, filamentstrukturer og tværgående transport i høj grad bestemmer både indeslutningens effektivitet og belastningen på reaktorens vægge. Hvis disse processer ikke forstås og beskrives tilstrækkeligt præcist, bliver det vanskeligt at optimere energibalancen i reaktoren [2, 3].

Randplasmaets dynamik er imidlertid vanskelig at analysere direkte, fordi fysikken beskrives gennem koblede, ikke lineære partielle differentialligninger. I dette projekt anvendes Hot Edge SOL Electrostatic (HESEL) modellen, som er en reduceret drift fluid model udviklet til at beskrive turbulens og transport i plasmaets rand og i SOL regionen [4, 3]. HESEL modellen er fysisk konsistent nok til at repræsentere centrale mekanismer i randplasmaet, men samtidig tilstrækkeligt reduceret til at kunne simuleres numerisk i praksis. Netop derfor er den et godt udgangspunkt for at undersøge, hvordan moderne datadrevne metoder kan kombineres med klassisk plasmafysisk modellering.

Den traditionelle tilgang til HESEL og lignende modeller er numerisk simulering, eksempelvis ved finite difference diskretisering i rum og tidsintegration af det resulterende ligningssystem. Denne tilgang er veletableret, men også beregningstung, og hvis man ønsker at sammenligne mange scenarier eller udforske store parameterrum, bliver fulde simuleringer hurtigt en flaskehals. Problemet er derfor ikke alene at kunne simulere systemet korrekt, men også at kunne gøre det effektivt og på en måde, der bevarer den væsentlige fysik.

Her bliver Scientific Machine Learning (SciML) interessant. SciML betegner metoder, hvor maskinlæring kombineres med kendt struktur fra naturvidenskabelige modeller, eksempelvis differentialligninger og bevarelseslove. For plasmafysik er det særligt lovende, fordi mange problemer netop er kendetegnet ved dyre simuleringer og stærke fysiske begrænsninger, som modellerne bør respektere. En vellykket SciML model kan derfor potentielt fungere som en hurtigere surrogatmodel i udvalgte arbejdsgange.

Projektets motivation er først og fremmest metodisk: hvis SciML kan anvendes meningsfuldt på HESEL modellen, peger det mod en mere fleksibel måde at arbejde med komplekse plasmamodeller på. Projektet placerer sig derfor i krydsfeltet mellem anvendt matematik, numerisk analyse, plasmafysik og maskinlæring, og netop dette tværfaglige spændingsfelt gør undersøgelsen fagligt relevant.

1.2 Formål og forskningsspørgsmål

Formålet med rapporten er at undersøge, hvordan SciML kan anvendes til modellering af raddynamik i plasma med udgangspunkt i HESEL modellen, og i hvilken grad sådanne modeller kan supplere eller effektivisere klassiske numeriske simuleringer. Udgangspunktet er ikke, at maskinlæring skal erstatte fysikbaserede metoder, men at afklare, hvornår en fysikinformeret læringsmodel er et nyttigt redskab.

Derfor stilles følgende forskningsspørgsmål:

1. Hvordan kan SciML, særligt fysikinformerede neurale netværk, anvendes til modellering af HESEL ligningssystemet, og hvordan sammenlignes løsningerne med klassiske numeriske simuleringer
2. Hvilken betydning har det for modellens kvalitet, om man vægter de Partielle Differential ligninger (PDE)'er og data'en forskelligt, og hvordan påvirker dataopløsningen resultaterne?
3. I hvor høj grad kan modellen generalisere til ukendte punkter, tidsskridt og simulationskonfigurationer
4. Hvor meget kan modellen effektiviseres, før tabet i præcision bliver væsentligt, og hvordan bør beregningstid, hukommelsesforbrug og nøjagtighed afvejes i praksis?

Disse spørgsmål er valgt, fordi de tilsammen dækker både den fysiske og den beregningsmæssige side af problemet. Det er ikke tilstrækkeligt, at en model blot kan reproducere enkelte løsninger; den skal også være robust, fortolkbar og anvendelig i situationer, hvor man ønsker mere end ren interpolation. Derfor handler projektet både om modelleringsevne og om videnskabelig troværdighed.

Rapporten har samtidig et læringsperspektiv. Arbejdet forudsætter forståelse af numerisk løsning af PDE systemer, grundlæggende plasmafysik i randregionen og udvikling samt evaluering af SciML modeller på dynamiske systemer.

Resten af rapporten er struktureret som følger. Først præsenteres den fysiske baggrund for HESEL modellen og de numeriske metoder, der anvendes til simulering af ligningssystemet. Herefter beskrives implementeringen af eksperimenterne og den måde, data genereres og behandles på. Dernæst analyseres resultaterne fra både de klassiske simuleringer og de anvendte SciML metoder med henblik på at vurdere nøjagtighed, generalisering og effektivitet. Afslutningsvis diskuteres projektets væsentligste resultater, begrænsninger og perspektiver for fremtidigt arbejde.

2 Metode

Dette kapitel gennemgår den teoretiske og metodiske baggrund for projektet. Først introduceres approksimation af PDE'er med neurale netværk, derefter HESEL modellen og de numeriske metoder, og til sidst den eksperimentelle tilgang.

2.1 Approksimation af PDE'er med neurale netværk

Neurale netværk kan bruges til at approksimere løsningen til en differentialligning ved at repræsentere den ukendte løsning som en differentiabel funktion med et sæt justerbare parametre. Da simuleringerne og det neurale netværk begge er diskretiseret i tid og rum kan det neurale netværk indsættes i den approksimerede løsning og derefter justere parametrene, så funktionen både opfylder differentialligningen og de tilhørende rand eller initialbetingelser [5].

For et generelt PDE problem kan løsningen skrives som

$$\mathcal{F}(t, x, y, u, \partial_t u, \nabla u, \nabla^2 u) = 0, \quad (t, x, y) \in [0, T] \times \Omega, \quad (1)$$

med randbetingelser

$$\mathcal{B}[u] = g \quad \text{på } \partial\Omega \quad (2)$$

og eventuelt en initialbetingelse

$$u(0, x, y) = u_0(x, y). \quad (3)$$

Metoden erstatter den ukendte løsning med en approksimation $u_\theta(t, x, y)$, hvor θ samler de frie parametre. Ved at indsætte u_θ i ligning (1) fås residualet

$$r_\theta(t, x, y) = \mathcal{F}(t, x, y, u_\theta, \partial_t u_\theta, \nabla u_\theta, \nabla^2 u_\theta). \quad (4)$$

Hvis u_θ var den eksakte løsning, ville residualet være nul overalt i domænet. Træningen består derfor i at vælge θ , så residualet bliver lille i domænet.

2.1.1 Hårde begrænsninger for randbetingelser

En klassisk tilgang er at indbygge rand og initialbetingelser direkte i ansatsen for løsningen. I formuleringen fra [5] skrives den approksimerede løsning som

$$u_\theta(t, x, y) = A(t, x, y) + M(t, x, y)N_\theta(t, x, y), \quad (5)$$

hvor A vælges, så den alene opfylder rand og initialbetingelserne, mens maskefunktionen M konstrueres, så den forsvinder på de dele af randen, hvor Dirichletbetingelser pålægges. Dermed påvirker korrektionsleddet MN_θ ikke de påtvungne randværdier. Ved blandede randbetingelser kan M tilsvarende vælges, så også den normale afledte forsvinder på de relevante randområder.

For en elliptisk ligning, eksempelvis Poisson ligningen

$$\nabla^2 u(x, y) = f(x, y), \quad (x, y) \in \Omega,$$

kan residualet evalueres direkte fra de rumlige afledte af u_θ . For tidsafhængige PDE'er behandles tiden blot som endnu en inputvariabel, så både $\partial_t u_\theta$ og de rumlige afledte indgår i residualet. For et koblet PDE system kan approksimationen tilsvarende skrives som en vektor

$$\mathbf{u}_\theta(t, x, y) \in \mathbb{R}^m,$$

hvor alle felter trænes samtidig ved at minimere residualerne for hele systemet.

Når randbetingelserne er indbygget på denne måde, behøver tabsfunktionen kun at håndhæve selve differentiaalligningen, og en typisk tabsfunktion kan derfor skrives som

$$\mathcal{L}(\theta) = \frac{1}{N_r} \sum_{i=1}^{N_r} |r_\theta(t_i, x_i, y_i)|^2, \quad (6)$$

hvor $\{(t_i, x_i, y_i)\}_{i=1}^{N_r}$ er det diskretiseret domæne. Her bliver randbetingelserne hårdt håndhævet, fordi de opfyldes præcist ved konstruktion af ansatsen.

2.1.2 Fysik informerede neurale netværk og bløde begrænsninger

I fysikinformerede neurale netværk (Physics Informed Neural Networks, PINNs) bruges den samme grundidé, nemlig at approksimere den ukendte løsning med et neuralt netværk og evaluere differentiaalligningen gennem et residual. I stedet for at indbygge rand og initialbetingelser eksplicit i ansatsen vælges der dog ofte en mere direkte parameterisering af løsningen, og betingelserne håndhæves i tabsfunktionen. Denne formulering gennemgås i *Physics Informed Deep Learning* af Raissi, Perdikaris og Karniadakis [6].

For et tidsafhængigt PDE problem på formen

$$\partial_t u + \mathcal{F}[u] = 0, \quad (7)$$

hvor $\mathcal{F}$ er en ikke lineær differentialoperator, introduceres både en approksimation $u_\theta(t, x, y)$ og et tilhørende fysikresidual

$$f_\theta(t, x, y) = \partial_t u_\theta(t, x, y) + \mathcal{F}[u_\theta](t, x, y). \quad (8)$$

De samme parametre θ indgår i både u_θ og f_θ , så optimeringen samtidig former løsningen og dens afledte egenskaber.

I denne tilgang opdeles tabsfunktionen i et dataled og et fysikled,

$$\mathcal{L}(\theta) = \mathcal{L}_u + \mathcal{L}_f, \quad (9)$$

hvor

$$\mathcal{L}_u = \frac{1}{N_u} \sum_{i=1}^{N_u} |u_\theta(t_u^i, x_u^i, y_u^i) - u^i|^2 \quad (10)$$

måler afvigelsen fra kendte initial og randdata, mens

$$\mathcal{L}_f = \frac{1}{N_f} \sum_{i=1}^{N_f} |f_\theta(t_f^i, x_f^i, y_f^i)|^2 \quad (11)$$

måler, hvor godt differentialligningen opfyldes i det indre domæne.

Her angiver $\{(t_u^i, x_u^i, y_u^i), u^i\}_{i=1}^{N_u}$ de punkter, hvor løsningens værdi er kendt fra initial eller randbetingelser, mens $\{(t_f^i, x_f^i, y_f^i)\}_{i=1}^{N_f}$ er de punkter, hvor fysikken håndhæves gennem residualen.

Denne formulering svarer til blødt begrænset randbetingelse, fordi rand og initialbetingelser ikke nødvendigvis opfyldes præcist i hvert punkt, men i stedet gøres dyre at bryde gennem tabsfunktionen. Hertil er metoden fleksibel, fordi samme grundstruktur kan anvendes til mange forskellige typer PDE'er uden at skulle konstruere en særlig ansatz for hver ny kombination af randbetingelser. Fysikleddet $\mathcal{L}_f$ virker samtidig som en regularisering, der begrænser løsningsrummet til funktioner, som i høj grad respekterer den underliggende differentialligning [6].

2.1.3 Valg af rand begrænsning

Som udgangspunkt anvendes almindelige fuldt forbundne feedforward netværk. Dette er i tråd med både den klassiske formulering i [5] og PINN formuleringen i [6], hvor løsningen repræsenteres ved et relativt simpelt neuralt netværk frem for en mere specialiseret arkitektur. Valget af model handler derfor her primært om, hvordan input og output organiseres, og hvordan randbetingelserne håndhæves.

For modellen PINN bruges en punktvise model. Inputtet består af de to foregående tidssteg for et lokalt 3×3 naboskab omkring ét punkt samt punktets fysiske koordinater, hvor netværket forudsiger de fire felter i næste tidssteg for centerpunktet. Denne opsætning følger den bløde PINN formulering fra [6], fordi netværkets output bruges direkte som approksimation af løsningen, mens differentialligning, initialbetingelser og randbetingelser håndhæves gennem tabsfunktionen. PINN er derfor blødt begrænsning i randen, fordi betingelserne kun opfyldes i det omfang optimeringen gør det billigt at bryde dem.

For modellen PINN_z bruges stadig et feedforward netværk, men nu på linjeform. Inputtet er de to foregående tidssteg for hele x akse og et lille vindue i z retningen samt deres koordinater, netværket forudsiger de fire felter for hele x akse i næste tidssteg. Den rå netværksprædiction indsættes derefter i en hårdt begrænset indpakning på formen

$$u_\theta = g_x + \sigma_t(H_x - g_x + \mathcal{C}[N_\theta]), \quad (12)$$

hvor H_x og korrektionsleddet $\mathcal{C}[N_\theta]$ konstrueres ud fra randbetingelserne, så korrektionsleddet forsvinder på randen, svarende til idéen i [5]. PINN_z er derfor hårdt begrænset i randen,

fordi randbetingelserne bygges direkte ind i selve outputkonstruktionen i stedet for kun at blive straffet i tabsfunktionen.

Denne opdeling er valgt, fordi punktmodellen er fleksibel og velegnet til den klassiske blødt begrænset PINN formulering, mens linjemodellen gør det praktisk at indbygge randbetingelser eksplicit langs hele x akse. Derefter udføres en separat udforskning af lagdybde, hidden dimension og aktiveringsfunktioner for at vælge den endelige arkitektur.

2.2 Fysikken bag HESEL modellen

HESEL modellen (Hot Edge SOL Electrostatic) er en reduceret plasma model udviklet til at beskrive turbulens og transportfænomener i plasmaets rand i en tokamak. Særligt fokuserer modellen på overgangsområdet mellem det indelukkede plasma og scrape off layer (SOL), hvor magnetfeltlinjerne ikke længere er lukkede. I dette område transporteres partikler og energi på tværs af magnetfeltlinjerne gennem turbulente strukturer, kaldet blobs. HESEL modellen er baseret på drift fluid approksimationen af Braginskii's plasma fluidligninger[4].

HESEL bliver reduceret, fra Braginskii's ligninger, ved at udnytte de karakteristiske tids og længdeskalaer i plasmaets rand, hvilket vedligeholder simulering fysisk realitet og numerisk stabilitet.

I dette afsnit gennemgås de grundlæggende trin i Braginskii's udledning, hvorefter den endelige HESEL model præsenteres. Formålet er ikke at give en fuldstændig fysisk redegørelse for HESEL modellen, men derimod at introducere de centrale idéer og ligninger, som modellen bygger på.

2.2.1 Braginskii's fluidligninger

Braginskii's plasma fluid ligninger er et sæt makroskopiske ligninger, der beskriver bevarelsen af partikler, impuls og energi for en plasmaart. Som udgangspunkt anvendes Boltzmann ligningen, som beskriver udviklingen af plasmaets fordelingsfunktion i fase rummet. Faserummet består af både partiklernes position $\mathbf{r}$ og hastighed $\mathbf{v}$. For et kollisionsfrit system gælder

$$\frac{df_a}{dt} = 0.$$

Anvendes kædereolen i faserummet fås:

$$\frac{df_a}{dt} = \frac{\partial f_a}{\partial t} + \frac{dx_\beta}{dt} \frac{\partial f_a}{\partial x_\beta} + \frac{dv_\beta}{dt} \frac{\partial f_a}{\partial v_\beta}.$$

Herefter sættes:

$$\begin{aligned}\frac{dx_\beta}{dt} &= v_\beta, \\ \frac{dv_\beta}{dt} &= \frac{F_{a\beta}}{m_a}\end{aligned}$$

Når kollisioner inkluderes opnås Boltzmann ligningen

$$\frac{\partial f_a}{\partial t} + \frac{\partial}{\partial x_\beta} (v_\beta f_a) + \frac{\partial}{\partial v_\beta} \left(\frac{F_{a\beta}}{m_a} f_a \right) = C_a \quad (13)$$

Her er β indeks for koordinater i det fysiske rum. Det første led, $\partial_t f_a$, beskriver den lokale tidslige ændring af fordelingsfunktionen. Det andet led, $\partial_{x_\beta} (v_\beta f_a)$, beskriver transport i det fysiske rum. Det tredje led, $\partial_{v_\beta} \left(\frac{F_{a\beta}}{m_a} f_a \right)$, beskriver transport i hastighedsrummet. Højresiden C_a beskriver ændringen i fordelingsfunktionen som følge af kollisioner mellem partikler.

I ligningen indgår følgende størrelser:

$f_a = f_a(t, \mathbf{r}, \mathbf{v})$	fordelingsfunktion for partikelart a ,
$\mathbf{F}_a = e_a \mathbf{E} + \frac{e_a}{c} (\mathbf{v} \times \mathbf{B})$	Lorentzkraften på partikelart a ,
m_a	massen af partikelart a ,
e_a	ladningen af partikelart a ,
C_a	kollisionsleddet for partikelart a .

Her beskriver $f_a(t, \mathbf{r}, \mathbf{v}) d\mathbf{v} d\mathbf{r}$ tætheden af partikler af art a i det fysiske rum $d\mathbf{r}$ omkring punktet $\mathbf{r}$. I resten af udledningen undlades det at specificere hvilken partikelart, der er tale om på samme måde som Braginskii gør. Da kollisioner over alle hastigheder ikke ændrer det totale antal partikler gælder for kollisions termet.

$$\int C d\mathbf{v} = 0,$$

Herefter defineres følgende makroskopiske størrelser:

$$\begin{aligned}n(\mathbf{r}, t) &= \int f(\mathbf{r}, \mathbf{v}, t) d\mathbf{v} \\ V_a(\mathbf{r}, t) &= \frac{1}{n} \int \mathbf{v} f(\mathbf{r}, \mathbf{v}, t) d\mathbf{v} = \langle v \rangle \\ T(\mathbf{r}, t) &= \frac{m}{3} \langle (v - V)^2 \rangle\end{aligned}$$

Her er n , V og T makroskopiske størrelser for tætheden, middelhastigheden og temperaturen, for partikler i punktet $\mathbf{r}$ til tiden t . For at komme frem til Braginskii's 3 ligninger, der beskriver tæthed, impuls og energi, ganges (13) med 1, mv og $\frac{1}{2}mv^2$. Herefter integreres ligningerne over hele hastighedsrummet, hvorefter n , V og T indsættes i ligningerne. Herved

fremkommer Braginskii's ligninger:

$$\frac{\partial n}{\partial t} + \nabla \cdot (n\mathbf{V}) = 0 \quad (14)$$

$$\frac{\partial}{\partial t} (mn\mathbf{V}) + \frac{\partial}{\partial x_\beta} (mn\langle vv_\beta \rangle) - en \left(E + \frac{1}{c} [\mathbf{V}\mathbf{B}] \right) = \int m\mathbf{v} C d\mathbf{v} \quad (15)$$

$$\frac{\partial}{\partial t} \left(\frac{mn}{2} \langle v^2 \rangle \right) + \nabla \cdot \left(\frac{mn}{2} \langle v^2 \mathbf{v} \rangle \right) - en\mathbf{E}\mathbf{V} = \int \frac{mv^2}{2} C d\mathbf{v} \quad (16)$$

(Selve integrationen og udledningen af ligningerne er ikke udført i denne rapport.) Ligning (14) kaldes kontinuitetsligningen og beskriver bevarelsen af partikler. Ligningerne (15) og (16) kaldes henholdsvis impuls og energiligningen og beskriver bevarelsen af henholdsvis impuls og energi i plasmaet. (15) og (16) bliver yderligere reduceret.

For hastigheden af partikeltyper defineres middel hastighed $\mathbf{V}$ og tilfældig hastighed, $\mathbf{v}' = \mathbf{v} - \mathbf{V}$. Med dette udledes:

$$\begin{aligned} \langle v_\alpha v_\beta \rangle &= \langle (V_\alpha + v'_\alpha)(V_\beta + v'_\beta) \rangle \\ &= V_\alpha V_\beta + V_\alpha \langle v'_\beta \rangle + V_\beta \langle v'_\alpha \rangle + \langle v'_\alpha v'_\beta \rangle \end{aligned}$$

Da den tilfældige hastighed er defineret som afvigelsen fra middelhastigheden,

$$v'_\alpha = v_\alpha - V_\alpha$$

gælder

$$\langle v'_\alpha \rangle = \langle v'_\beta \rangle = 0$$

Dermed fås

$$\langle v_\alpha v_\beta \rangle = V_\alpha V_\beta + \langle v'_\alpha v'_\beta \rangle$$

For at simplificere Braginskii's impuls ligning (15) defineres følgende:

$$\frac{df}{dt} = \frac{\partial f}{\partial t} + V_\beta \frac{\partial f}{\partial x_\beta} = \frac{\partial f}{\partial t} + (\mathbf{V}\nabla)f \quad (17)$$

$$p = \frac{mn\langle v'^2 \rangle}{3} = nT \quad (18)$$

$$\pi_{\alpha\beta} = nm\langle v'_\alpha v'_\beta - \frac{1}{3}v'^2\delta_{\alpha\beta} \rangle \quad (19)$$

$$\mathbf{R} = \int m\mathbf{v}' C d\mathbf{v} \quad (20)$$

Operatoren (17) kaldes substantive afledte og er anvendt på funktionen f . Ligning (18) definerer det isotrope tryk, (19) den viskøse spændingstensor der beskriver afvigelser fra isotropt tryk, (20) beskriver netto momentumoverførsel som følge af kollisioner mellem partikler.

Braginskii's impuls ligning kan skrives på formen:

$$mn \frac{dV_a}{dt} = -\frac{\partial p}{\partial x_a} - \frac{\partial \pi_{a\beta}}{\partial x_\beta} + en \left(E_a + \frac{1}{c} [\mathbf{VB}]_a \right) + R_a \quad (21)$$

Venstresiden, af Braginskii's momentum ligning(21), repræsenterer ændringen i plasmaets momentum. Højresiden består af kræfter fra trykgradienter, viskøse spændinger, elektromagnetiske felter og kollisioner med andre partikler. Disse processer styrer transporten og udviklingen af plasmaets makroskopiske bevægelse.

Braginskii's energi ligning (16) er også ønsket at skrive på en mere kompakt form. Til dette udledes følgende størrelser:

$$\begin{aligned} \left\langle \frac{v^2}{2} v_\beta \right\rangle &= \frac{1}{2} V^2 V_\beta + V_\alpha \langle v'_\alpha v'_\beta \rangle + \frac{1}{2} \langle v'^2 \rangle V_\beta + \left\langle \frac{v'^2}{2} v'_\beta \right\rangle \\ &= \left(\frac{1}{2} V^2 + \frac{5P}{2mn} \right) V_\beta + \frac{1}{mn} V_\alpha \pi_{\alpha\beta} + \left\langle \frac{1}{2} v'^2 v'_\beta \right\rangle \end{aligned}$$

Ved at introducere følgende definitioner:

$$\begin{aligned} Q &= \int \frac{m}{2} v'^2 C d\mathbf{v} \\ \mathbf{q} &= \int \frac{m}{2} v'^2 \mathbf{v}' f d\mathbf{v} = \left\langle \frac{1}{2} m v'^2 \mathbf{v}' \right\rangle \end{aligned}$$

Hvor Q beskriver varme genereret af kollisioner mellem partikler. $\mathbf{q}$ er flux tætheden for tilfældig transport. Herfra kan Braginskii's energi ligning komme på den kompakte form:

$$\frac{\partial}{\partial t} \left(\frac{nm}{2} V^2 + \frac{3}{2} nT \right) + \frac{\partial}{\partial x_\beta} \left(\left(\frac{mn}{2} V^2 + \frac{5}{2} nT \right) V_\beta + (\pi_{\alpha\beta} V_\alpha) + q_\beta \right) = en \mathbf{E} \mathbf{V} + \mathbf{R} \mathbf{V} + Q \quad (22)$$

Det første led repræsenterer ændringen i den totale energi, som består af summen af den kinetiske energi og den interne termiske energi. Det andet led beskriver energifluxen og indeholder bidrag fra konvektiv energitransport, viskøse spændinger og varmeledning. Højresiden repræsenterer arbejdet udført af ydre kræfter samt varmeproduktion og energiudveksling forårsaget af kollisioner.

Herved udlede Braginskii sine tre ligninger for tæthed, impuls og energi udvikling for

plasma [7].

Braginskii's kontinuitetsligning

$$\frac{\partial n}{\partial t} + \nabla \cdot (n\mathbf{V}) = 0 \quad (23)$$

Braginskii's momentumligning

$$mn \frac{dV_a}{dt} = -\frac{\partial p}{\partial x_a} - \frac{\partial \pi_{a\beta}}{\partial x_\beta} + en \left(E_a + \frac{1}{c} [\mathbf{VB}]_a \right) + R_a \quad (24)$$

Braginskii's energitransport ligning

$$\frac{\partial}{\partial t} \left(\frac{nm}{2} V^2 + \frac{3}{2} nT \right) + \frac{\partial}{\partial x_\beta} \left(\left(\frac{nm}{2} V^2 + \frac{5}{2} nT \right) V_\beta + (\pi_{\alpha\beta} V_\alpha) + q_\beta \right) = en\mathbf{E}\mathbf{V} + \mathbf{R}\mathbf{V} + Q \quad (25)$$

Ved modellering af turbulens i randplasmaer er Braginskii's ligninger imidlertid ofte for komplekse til direkte numerisk anvendelse. Derfor introduceres den reducerede drift fluid HESEL modeller, hvor en række fysiske antagelser anvendes til at forenkle beskrivelsen af plasmaets dynamik.

2.2.2 HESEL modellen

I dette projekt anvendes HESEL modellen, som er en n dimensionel drift fluid model udviklet til at beskrive turbulens og transport i tokamak plasmaers rand og SOL regioner. I dette projekt simuleres den i 2 dimensioner. Modellen bygger på Braginskii's fluidbeskrivelse og fremkommer gennem en række approksimationer og reduktioner af de oprindelige ligninger. En detaljeret udledning ligger uden for dette projekts omfang og kan findes i litteraturen [4]. Fokus i dette projekt er derfor ikke på den fulde udledning af HESEL modellen, men på anvendelsen af modellen som fysisk grundlag for de efterfølgende numeriske og maskinlæringsbaserede metoder.

Notationen i HESEL modellen kommer til at følge tæt notationen i [3] og [2]:

$$d_t n + n\mathcal{K}(\phi) - \mathcal{K}(p_e) = \Lambda_n \quad (26)$$

$$d_t^0 \omega^* + \{\phi, p_i\} - \mathcal{K}(p_e + p_i) = \Lambda_{\omega^*} \quad (27)$$

$$\frac{3}{2} d_t p_e + \frac{5}{2} p_e \mathcal{K}(\phi) - \frac{5}{2} \mathcal{K} \left(\frac{p_e^2}{n} \right) = \Lambda_{p_e} \quad (28)$$

$$\frac{3}{2} d_t p_i + \frac{5}{2} p_i \mathcal{K}(\phi) + \frac{5}{2} \mathcal{K} \left(\frac{p_i^2}{n} \right) - p_i \mathcal{K}(p_e + p_i) = \Lambda_{p_i} \quad (29)$$

Det der gør HESEL modellen numerisk effektiv og samtidig fysisk realistisk er, at de tværgående komponenter af momentumligningen reduceres til velkendte plasma drifter. Særligt beskrives plasmaets bevægelse i det tværgående plan ved $E \times B$ driften, mens inertielle effekter indgår gennem den modificerede vorticitet. Derved undgås en direkte løsning af den fulde momentumligning, hvilket reducerer den beregningsmæssige kompleksitet betydeligt.

Den første ligning (26) beskriver bevarelsen af plasmaets partikeltæthed. Ledet $n\mathcal{K}(\phi)$ repræsenterer transport som følge af elektriske felter i et krumt magnetfelt, mens $\mathcal{K}(p_e)$ beskriver drivningen fra elektronstryksgradier. Ligningen udgør dermed grundlaget for beskrivelsen af partikeltransport og dannelsen af tæthedsfluktuationer i HESEL modellen.

Vorticitetsligningen (27) beskriver udviklingen af plasmaets tværgående rotationsbevægelse. Ligningen fremkommer i drift fluid formuleringen ved at kombinere ion og elektronkontinuitetsligningerne under antagelsen om kvasi neutralitet. Den modificerede vorticitet ω^* indeholder både $E \times B$ vorticitet og bidrag fra iontrykket.

Poisson klammen $\{\phi, p_i\}$ beskriver vekselvirkningen mellem det elektrostatiske potentiale og iontrykket, mens krumningsleddet $\mathcal{K}(p_e + p_i)$ repræsenterer drivningen af turbulens gennem trykgradier i det krumme magnetfelt. Højresiden indeholder viskøse og kollisionsbestemte dissipative mekanismer. Ligningen er central for beskrivelsen af turbulens og filamentdynamik i HESEL.

Elektron (28) og iontryk (29) ligningerne beskriver udviklingen af plasmaets termiske energi gennem henholdsvis elektrontrykket p_e og iontrykket p_i . Ligningerne er afledt fra Braginskiis energitransportligninger og indeholder transport med plasmaflowet, kompression i det krumme magnetfelt samt energitransport drevet af trykgradier. Samtidig beskriver de energiudveksling mellem plasmaets termiske energi og kollektive bevægelse.

Lambda termene er fra artikelen [3] og her er de givet ved.

$$\Lambda_n = D_e \nabla_{\perp}^2 n - \alpha \left(\tilde{T}_e + \frac{\bar{T}_e}{\bar{n}} \tilde{n} - \tilde{\phi} \right) - \frac{n}{\tau} - \frac{n - n_p}{\tau_p} \quad (30)$$

$$\Lambda_{\omega}^* = D_i \nabla_{\perp}^2 \omega^* - \alpha \left(\tilde{T}_e + \frac{\bar{T}_e}{\bar{n}} \tilde{n} - \tilde{\phi} \right) - \frac{\omega^*}{\tau} + \frac{\rho_s}{L_c} \left[1 - \exp \left(\phi_m - \frac{\phi_s}{T_{e,s}} \right) \right] \quad (31)$$

$$\begin{aligned} \Lambda_{p_e} = & \frac{5}{2} D_e \nabla_{\perp}^2 p_e - \alpha \bar{T}_e \left(\tilde{T}_e + \frac{\bar{T}_e}{\bar{n}} \tilde{n} - \tilde{\phi} \right) - \frac{9p_e}{2\tau} - \frac{p_e - p_{e,p}}{\tau_p} \\ & + \left(\frac{16}{6} - \frac{5}{2} \right) \nabla \cdot (n \nabla_{\perp} T_e) - \frac{T_e}{\tau^{SH}} - \Theta \end{aligned} \quad (32)$$

$$\Lambda_{p_i} = D_i \nabla_{\perp}^2 p_i - D_i T_i \nabla_{\perp}^2 n - \frac{9p_i}{2\tau} + \Theta - \frac{p_i - p_{i,p}}{\tau_p} + p_i \Lambda_{\omega^*} \quad (33)$$

Λ koefficienterne står for at beskrive kollisionsprocesserne, varmeledning, viskøse effekter og parallelle tab og virker dissipativt på systemet. Her repræsenterer D_e og D_i de neoklassiske tværgående transportkoefficienter, som beskriver diffusiv udjævning af henholdsvis partikel, ion, elektron og vorticitetsprofiler. Koefficienten α parametriserer koblingen til parallel driftbølge dynamik på lukkede feltlinjer og driver transport gennem forskelle mellem tæthed, temperatur og potentiale. τ og τ_p beskriver hvor hurtigt feltet tvinges mod profilen, funktionerne med subskript p , beskriver parallelle tab: τ er den karakteristiske tidsskala for tab til SOL og sheath, mens τ_p angiver den tidsskala, hvormed plasmaet styres mod de påtvungne kildeprofiler i randregionen. For dette eksperiment er profilerne givet ved:

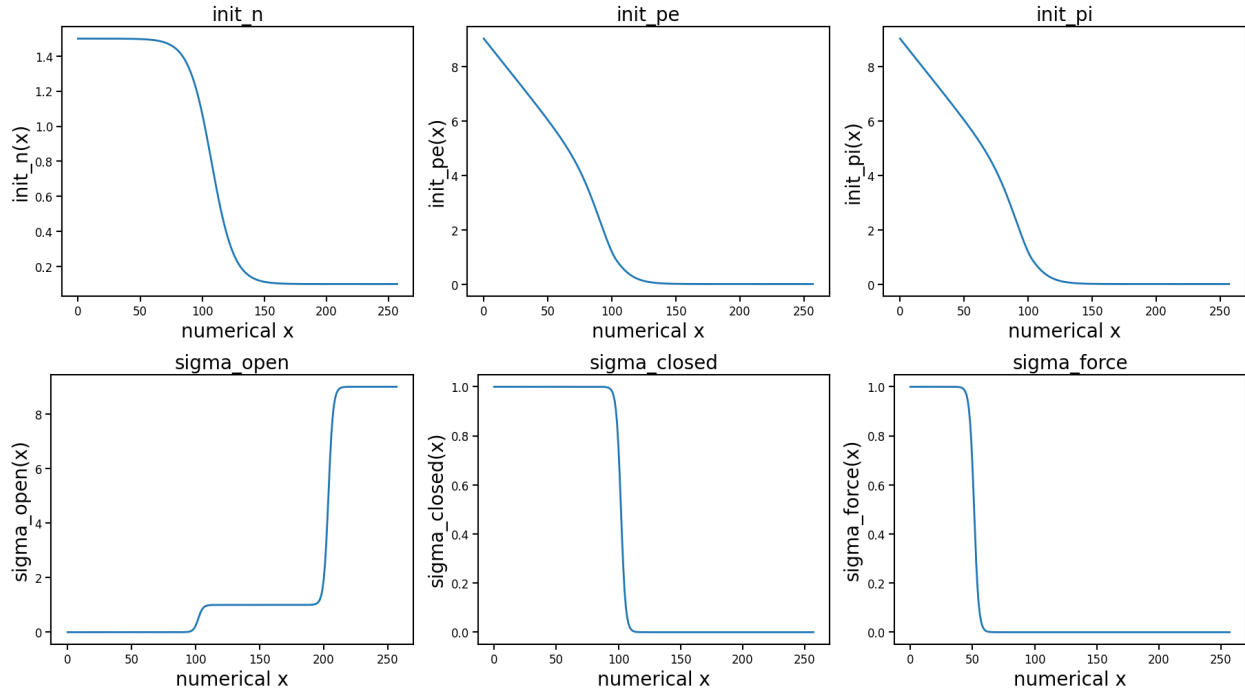


Figure 1: Påtvungne profiler i numerisk domæne, med ghost cells, "init" er n_p , $p_{e,p}$ og $p_{i,p}$, med sigma profiler der bestemmer hvor Λ termene håndhæves. x-aksen har Bohm normaliseret længde: $x = 140/\rho_s$

Funktionerne i HESEL modellen er givet ved:

$$n : n(x, y, t) \quad (34)$$

$$p_{e,i} : p_{e,i}(x, y, t) \quad (35)$$

$$T_{e,i} : T_{e,i}(x, y, t) \quad (36)$$

$$p_{e,i} = nT_{e,i} \quad (37)$$

$$\phi : \phi(x, y, t) \quad (38)$$

$$\omega^* : \omega^*(x, y, t) = \nabla^2 \phi + \nabla^2 p_i \quad (39)$$

Her beskriver n plasmaets partikel tæthed, $p_{e,i}$ er henholdsvis elektron og iontryk, $T_{e,i}$ er elektron og ion temperatur, ϕ er det elektriske potentiale og ω^* er den modificerede vorticitet. Sammenhængen mellem tæthed, tryk og temperatur, er givet ved $p_{e,i} = nT_{e,i}$, hvilket er følge af Boltzmanns konstanten og temperatur samles til et led

$$pV = N\kappa_b T, \quad pV = NT$$

da N er antallet af partikler kan tætheden n udtrykkes på følgende form:

$$p = \frac{N}{V}T, \quad p = nT$$

Når HESEL modellen simuleres, udføres det i Bohm normaliserede enheder, hvilket reducerer antallet af fysiske parametre. Dette gør simuleringen mere numerisk stabil og gør det muligt at beskrive plasmaets dynamik i en dimensionsløs form uafhængigt af de valgte referenceværdier. Bohm normalisering betyder at visse variabler ganges med konstanter for at gøre dem dimensionsløse.

$$t \rightarrow \omega_{ci,0}t, \quad x \rightarrow \frac{x}{\rho_s}, \quad n \rightarrow \frac{n}{n_0}, \quad T_{e,i} \rightarrow \frac{T_{e,i}}{T_{e0}}, \quad p_{e,i} \rightarrow \frac{p_{e,i}}{n_0 T_{e0}},$$

Det bemærkes at i literaturen bruges (x, y, z) koordinat indeksering, men når HESEL simuleres bruges (x, z, y) indeksering. I metode afsnittet vil (x, y, z) koordinat indeksering bruges men i 3 bruges (x, z, y) indeksering. Dette ændre ikke på noget fysisk eller matematisk men blot notationen.

Til sidst introduceres den række operatorer, som anvendes i HESEL modellen for at gøre ligningerne mere kompakte og læsbare.

$$\mathcal{K}(f) = -\frac{\rho_s}{R} \partial_y f \quad (40)$$

$$\{f, g\} = \partial_x f \partial_y g - \partial_y f \partial_x g \quad (41)$$

$$d_t^0 f = \partial_t f + \{\phi, f\} \quad (42)$$

$$d_t f = \partial_t f + \frac{1}{B} \{\phi, f\} \quad (43)$$

$$\bar{f} = \frac{1}{L_y} \int_0^{L_y} f dy \quad (44)$$

$$\tilde{f} = f - \bar{f} \quad (45)$$

$$\nabla = \begin{pmatrix} \partial_x \\ \partial_y \\ \partial_z \end{pmatrix} \quad (46)$$

$$\nabla^2 = \partial_x^2 + \partial_y^2 + \partial_z^2 \quad (47)$$

$$\nabla_{\perp} = \begin{pmatrix} \partial_x \\ \partial_y \end{pmatrix} \quad (48)$$

$$\nabla_{\perp}^2 = \partial_x^2 + \partial_y^2 \quad (49)$$

$$(50)$$

De sidste parametre er alle skalarer der bestemmer de geometriske, numeriske og fysiske størrelser. Disse kan læses om i [4]

2.3 Numeriske metoder

Simuleringsmotoren er bygget til numerisk at løse koblede partielle differentiaalligninger. I dette projekt anvendes et todimensionalt domæne, så PDE systemet kan skrives som

$$\frac{\partial \mathbf{u}}{\partial t} = \mathcal{F}(t, x, y, \mathbf{u}, \nabla \mathbf{u}, \nabla^2 \mathbf{u}), \quad (x, y) \in \Omega, \quad (51)$$

hvor $\mathbf{u}(t, x, y) \in \mathbb{R}^m$ repræsenterer ét felt eller et koblet system af m felter. Grundideen er at diskretisere rummet først og derefter udvikle den diskrete løsning i tid.

2.3.1 Rumlig diskretisering

Domænet vælges som et rektangel

$$\Omega = [x_{\min}, x_{\max}) \times [y_{\min}, y_{\max}),$$

og opdeles i et uniformt gitter med N_x punkter i x retningen og N_y punkter i y retningen. Gitterpunkterne kan skrives som

$$x_i = x_{\min} + i\Delta x, \quad y_j = y_{\min} + j\Delta y,$$

hvor

$$\Delta x = \frac{x_{\max} - x_{\min}}{N_x}, \quad \Delta y = \frac{y_{\max} - y_{\min}}{N_y}.$$

Den diskrete løsning betegnes ved $\mathbf{u}_{i,j}(t)$, som approksimerer den kontinuerte løsning i punktet (x_i, y_j) . For koblede systemer er $\mathbf{u}_{i,j}(t)$ en vektor. Når ghost cells indføres, lader man de fysiske gitterpunkter have indeks $i = 1, \dots, N_x$ og $j = 1, \dots, N_y$, mens de ekstra randceller får indeks 0 og $N_x + 1$ i x retningen samt 0 og $N_y + 1$ i y retningen.

Gitteret udvides med ghost cells omkring randen, så de samme stencilformler kan anvendes i hele det indre domæne. For periodiske randbetingelser kopieres værdierne fra den modsatte side, så der diskret gælder

$$\mathbf{u}_{0,j} = \mathbf{u}_{N_x,j}, \quad \mathbf{u}_{N_x+1,j} = \mathbf{u}_{1,j},$$

og tilsvarende i y retningen.

2.3.2 Numeriske differentialoperatorer

De rumlige afledte approksimeres med centrale differenser af anden orden. For et skalarfelt u fås

$$(\partial_x u)_{i,j} \approx \frac{u_{i+1,j} - u_{i-1,j}}{2\Delta x}, \quad (\partial_y u)_{i,j} \approx \frac{u_{i,j+1} - u_{i,j-1}}{2\Delta y}. \quad (52)$$

Tilsvarende approksimeres de andenordens afledte ved

$$(\partial_{xx} u)_{i,j} \approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{\Delta x^2}, \quad (\partial_{yy} u)_{i,j} \approx \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{\Delta y^2}. \quad (53)$$

Heraf følger den diskrete Laplace operator

$$(\nabla^2 u)_{i,j} \approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{\Delta x^2} + \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{\Delta y^2}.$$

Hvis PDE systemet indeholder advektionsled, skrives disse diskret som

$$\mathbf{v} \cdot \nabla u \approx v_x (\partial_x u)_{i,j} + v_y (\partial_y u)_{i,j}.$$

Denne diskretisering har truncationsfejl af størrelsesorden $\mathcal{O}(\Delta x^2 + \Delta y^2)$ for glatte løsninger.

Når alle rumlige operatorer er diskretiseret, kan PDE systemet omskrives til et stort system af ordinære differentiaalligninger,

$$\frac{d\mathbf{U}}{dt} = \mathbf{F}(t, \mathbf{U}), \quad (54)$$

hvor $\mathbf{U}(t)$ samler alle gitterværdier for alle felter.

2.3.3 Tidsintegration

Til tidsudviklingen anvendes den klassiske Runge–Kutta metode af fjerde orden, RK4. Givet den semidiskrete tilstand $\mathbf{U}^n \approx \mathbf{U}(t_n)$ og et tidsstep Δt_n beregnes

$$\begin{aligned} \mathbf{k}_1 &= \mathbf{F}(t_n, \mathbf{U}^n), \\ \mathbf{k}_2 &= \mathbf{F}\left(t_n + \frac{\Delta t_n}{2}, \mathbf{U}^n + \frac{\Delta t_n}{2} \mathbf{k}_1\right), \\ \mathbf{k}_3 &= \mathbf{F}\left(t_n + \frac{\Delta t_n}{2}, \mathbf{U}^n + \frac{\Delta t_n}{2} \mathbf{k}_2\right), \\ \mathbf{k}_4 &= \mathbf{F}(t_n + \Delta t_n, \mathbf{U}^n + \Delta t_n \mathbf{k}_3), \end{aligned}$$

hvorefter løsningen opdateres ved

$$\mathbf{U}^{n+1} = \mathbf{U}^n + \frac{\Delta t_n}{6} (\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4). \quad (55)$$

RK4 er en eksplicit metode med høj nøjagtighed, men stabiliteten afhænger stadig af de rumlige skalaer og styrken af de fysiske led.

Den samlede numeriske strategi er derfor et uniformt gitter med periodiske randbetingelser, centrale differensoperatorer i rummet, metode af linjer til at danne et stort ODE system og RK4 til tidsudviklingen.

2.4 Tilgang til eksperiment

Formålet med eksperimenterne er at undersøge, i hvilken grad fysikinformererede neurale netværk kan lære og reproducere dynamikken i plasmaets randregion, som beskrevet af HESEL modellen. Mere specifikt ønskes det at undersøge, hvor præcist en model kan ap-

proksimere udviklingen af systemets tilstand fra ét tidssteg til det næste. Givet en tilstand $u(t)$ er målet således at lære en approksimation af tidsudviklingsoperatoren

$$u(t + \Delta t) \approx \mathcal{F}_\theta(u(t)),$$

hvor $\mathcal{F}_\theta$ repræsenterer den lærte model med parametre θ . Hvis en sådan approksimation kan opnås med høj nøjagtighed, vil modellen potentielt kunne fungere som en surrogatmodel for dele af den numeriske løsning og dermed reducere behovet for kostbare beregninger.

HESEL modellen beskriver et stærkt ikke lineært og turbulent system, hvor små variationer i simulation konfigurationen kan føre til markant forskellige udviklingsforløb. Som følge heraf er det relevant først at undersøge modellens evne til at rekonstruere simuleringer genereret fra kendte simulerings konfigurationener. Dette giver et mål for, hvor godt modellen er i stand til at lære den underliggende dynamik i de data, den er trænet på.

Hvis modellen kan forudsige udviklingen af simuleringer, som ikke indgår i træningsdatasættet, indikerer det, at den har lært generelle strukturer i dynamikken frem for blot at huske specifikke løsninger. En sådan egenskab vil være særligt værdifuld i forbindelse med hurtig udforskning af nye parameterrum og scenarier, hvor numeriske simuleringer endnu ikke er udført.

Endelig undersøges modellens evne til at generalisere på tværs af rumlige opløsninger. Målet er at afklare, om en model trænet på et grovere gitter kan anvendes til at approksimere løsninger på finere gitter. En sådan skalerbarhed er særligt interessant i plasmafysiske anvendelser, hvor højopløselige simuleringer ofte udgør en væsentlig beregningsmæssig omkostning. Hvis modellen kan overføre information mellem forskellige opløsninger, kan dette potentielt reducere den samlede beregningstid og gøre det muligt at generere tilnærmede højopløselige løsninger betydeligt hurtigere end ved traditionelle numeriske metoder.

3 Implementering

Dette kapitel beskriver den konkrete implementering af projektets modeller, datastruktur og træningsopsætning. Fokus er på, hvordan metoderne fra det foregående kapitel er realiseret i kode og organiseret i projektets repository.

3.1 Overordnet struktur

Her gennemgås overordnet Github repositoryet for dette eksperiment [8]. `hesel_scraper` indeholder kode til at indlæse og behandle output fra BOUT++ og HESEL, samt til at opstille de fysiske residualer og randbetingelser. `SciML` indeholder de implementerede modeller og træningskode, herunder `PINN`, `PINN_z` og `PINO_z`. `sim_data` indeholder de genererede simulationsdatasæt, som bruges til træning, validering og test af modellerne. `skriftlig_arbajde` indeholder den skriftlige dokumentation for projektet, herunder rapport, projektplan, opsætning og øvrigt materiale.

3.2 Datahåndtering

`hesel_scraper` og `sim_data` udgør tilsammen projektets data bearbejdning. Folderen `hesel_scraper` fungerer som bindeled mellem rå HESEL simulationer og SciML modellerne, mens `sim_data` folderen indeholder selve simulationskørslerne.

De vigtigste filer i `hesel_scraper` kan opsummeres som:

- `hesel_scraper/bout_dump.py`: indeholder klassen `BOUTHESELInfo`, denne klasse læser og gemmer data fra `BOUT.settings` og `BOUT.dmp.*.nc` filer, og udleder de normaliserede parametre, der bruges videre i modellerne.
- `hesel_scraper/bout_phys.py`: indeholder klassen `BOUTHESELPhys`, som opstiller initialbetingelser, randbetingelser og ligningsresidualer for HESEL systemet.
- `hesel_scraper/api/dump_helper.py`: indeholder hjælpefunktioner til at fortolke og evaluere udtryk fra `BOUT.settings`.
- `hesel_scraper/api/operators.py`: indeholder autograd baserede differentialoperatorer, som bruges til at beregne gradienter og Laplaceoperatoren på modeloutputtet.

Hver undermappe svarer til en bestemt simulerings konfiguration med selve simulerings konfigurationen `BOUT.inp`. Efter simulering er startet generes blandt andet `BOUT.settings`, `BOUT.dmp.*.nc`, `BOUT.restart.*.nc` og `BOUT.log.*`. Dermed rummer hver mappe både simuleringsopsætningen og de tilhørende outputfiler, som efterfølgende behandles i `hesel_scraper`.

3.3 Modelimplimentering

SciML indeholder implementeringen af projektets læringsmodeller og den tilhørende træningskode. Mappen er opdelt i tre hovedretninger, som svarer til de tre modeltyper, der er undersøgt i projektet.

De vigtigste dele af SciML kan opsummeres som:

- SciML/PINN: indeholder den punktwise soft constraint model. Filen `træning.py` fungerer som indgangspunkt for træningen, `util/model.py` definerer selve MLP modellen, `util/data_loader.py` konstruerer de punktwise træningseksempler, og det hele samles files i `util/træning_helpers.py`, tabsfunktioner og checkpoint logik.
- SciML/PINN_z: indeholder den linjebaserede hard constraint model med MLP arkitektur. Filen `træning.py` styrer træningen, `utils/data_loader.py` opbygger linjebaserede input langs x akse, `utils/wrapper.py` indbygger randbetingelserne direkte i modeloutputtet, og `utils/training_helpers.py` håndterer træning, evaluering og lagring af checkpoints.
- SciML/PINO_z: indeholder den operatorbaserede hard constraint model. Strukturen ligner PINN_z, men i `utils/wrapper.py` bruges en FNO lignende model i stedet for et almindeligt MLP, mens `utils/data_loader.py` og `utils/training_helpers.py` varetager datahåndtering og træningsforløb på tilsvarende måde.

På tværs af disse mapper er strukturen derfor bevidst holdt ens: hver modeltype har et træningsscript, en dataloader og hjælpefunktioner til modelopbygning og træning. Det gør det lettere at sammenligne modeltyperne, fordi forskellene primært ligger i repræsentationen af input og output samt i, hvordan de fysiske begrænsninger håndhæves.

4 Data og eksperiment

Dette kapitel beskriver, hvordan datasættene til eksperimenterne er genereret og organiseret, samt hvordan de anvendes i de efterfølgende undersøgelser. Først gennemgås opsætning og generering af data, hvorefter udforskningen af modelarkitektur og hyperparametre præsenteres.

4.1 Opsætning og generering af data

4.1.1 BOUT opsætning

BOUT++ [9] og BOUT HESEL blev opsat på Danmarks Tekniske Universitet (DTU)'s High Performance Computing (HPC) cluster, i et separat softwaremiljø, hvor de nødvendige moduler for MPI, FFTW, NetCDF og Intel compiler først blev loadet. Dette var nødvendigt, fordi både kompilering og kørsel af simuleringerne afhænger af parallelisering og eksterne numeriske biblioteker. Derudover blev `OMP_NUM_THREADS` sat til 16, så opsætningen matchede den ønskede brug af clusterets CPU ressourcer. BOUT++ er selve simuleringsrammen og BOUT HESEL er simuleringsmodellen udviklet af DTU's Plasma Physics and Fusion Energy afdeling.

Selve installationen blev udført i to trin. Først blev BOUT++ klonet fra det officielle repository, sat til version `v4.3.2`, konfigureret med `./configure` og derefter kompileret med `make j16`. BOUT++ fungerer her som den generelle simuleringsramme, som håndterer numerisk løsning, gitterstruktur og parallel udførelse. Herefter blev BOUT HESEL klonet med tilhørende submoduler. I denne del af opsætningen blev de nødvendige komponenter fra `HESELCommon` koblet til projektet, så HESELmodellen kunne bygges oven på den allerede kompilerede BOUT++installation. Til sidst blev BOUT HESEL også kompileret med `make j16`.

Når både BOUT++ og BOUT HESEL var bygget, kunne simuleringerne køres med `mpirun np 4 ./hesel d <folder med BOUT.inp fil>`. Her læser programmet sin konfiguration fra den tilhørende `BOUT.inp`fil, som specificerer blandt andet gitteropløsning, tidsstep, randbetingelser og fysiske parametre. Denne opsætning gjorde det muligt at generere konsistente HESEL simuleringer på clusteret, som efterfølgende kunne bruges som datagrundlag for de læringsbaserede modeller. Dette projekts opsætning af BOUT++ og BOUT HESEL er dokumenteret i detaljer i filen `skriftlig_arbajde/opsætning`, hvor alle nødvendige kommandoer og trin er gennemgået for at sikre reproducerbarhed.

4.1.2 Data generering

Til dette er der generet 6 simuleringer. Herfra er 3 standard simuleringer og 3 simuleringer hvor eneste ændring er: tidsteppet, fysikken og grid opløsningen. Alle simuleringskonfigurationsfiler der bestemmer begyndelsestilstande, randbetingelser, grid opløsninger, fysiske parametre og tilfældige seed ligger i folderen `sim_data` i Github repositoryet.

- 3 standard simuleringer starter med `sim_data/Alexander_*` " Her er `*` en plads for forskellige seed "

- Folderen med modificeret tidsstep hedder `sim_data/Alexander__t_n`
- Folderen med modificeret fysik hedder `sim_data/Alexander__fys_R_1`
- Folderen med modificeret grid opløsning hedder `sim_data/Alexander__grid_c_0`

Alle input filer i `sim_data/Alexander_*/BOUT.inp`. Alle simuleringer blev udført på Danmarks Tekniske Universitet (DTU)'s High Performance Computing (HPC) cluster, med 16 CPU kerner. Hver simulering tog ikke længere end 1 time. Submit filen til simuleringen ligger her: `sim_data/submit_simulering.sh`. Træningen af modellerne blev udført lokalt. Derofr blev filer downloadet fra HPC clusteret til lokal maskine med

```
rsync -avh ssh:<sti til data> <sti til lokal folder>
```

Det er ikke forventet at download skulle ødelegge data'en når den komprimeres og downloades med `rsync`.

For simuleringen med modificeret tidsstep sættes variabelen `timestep = 25`. Det betyder at modellen skal forudsige længere frem i tiden end hvad den er trænet på.

For simuleringen med modificeret fysik er følgende variabler ændret til

```
Z_eff = 1.7
Bt = 2.32
Te0 = 75.6
Ti0 = 75.6
```

Disse blev vurderet til at være mest almindelige fysiske parametre at variere. For simuleringen med modificeret grid opløsning blev ændre til at være `nx = 514, nz = 512`

Alle filer, både standard og modificerede, havde forskellige seeds for at sikre at udviklingen af simuleringerne var forskellige. Dette gav følgende længder af simuleringer:

simulation	t_{end}	t_{idx}
pi	15015	1001
e	14055	937
phi_0	11130	742
_t_n	17950	718
_fys_R_1	7665	511
_grid_c_0	7155	477

Table 1: Længden af simuleringerne i Bohm normaliserede tidsenheder og antal tidsindekser.

Det bemærkes at alle standard simuleringerne og simuleringen med modificeret tidsstep var længere end simuleringerne med modificeret fysik og grid opløsning. Der blev generet flere filer med modificeret fysik og grid men dette var blot de simuleringer der kom længst.

4.1.3 Data visualisering

Selve visualiseringen af data er udført i `hesel_scraper/visualizer.ipynb`. Til indlæsning af simuleringsdata anvendes klassen `BOUTHESELInfo` fra biblioteket `bout_dump`. Klassen benytter `xbout` til at læse de parallelle outputfiler fra hver data folder, hvor simuleringsdomænet er opdelt i flere subdomæner, ét for hver processor. Disse subdomæner sammensættes automatisk til det oprindelige globale gitter, hvorefter data lagres som PyTorch tensorer.

Det første billede genereret er begyndelsestilstanden for hvert felt, som set i fig 1. Hvert felt er udvidet symmetrisk om zaksen. For felterne n , p_e og p_i tages logaritmen, og ω^* er initialiseret til 0 og ϕ er udregnet med formelen (39). For det første billede genereret tilføjes der også en lille tilfældig støj så alle simuleringer udvikler sig forskelligt over tid. Denne støj er ikke synlig i $t = 0$. Den vertikale bar ved siden af hver felt viser farveskalaen for det pågældende felt. Dette giver følgende billeder.

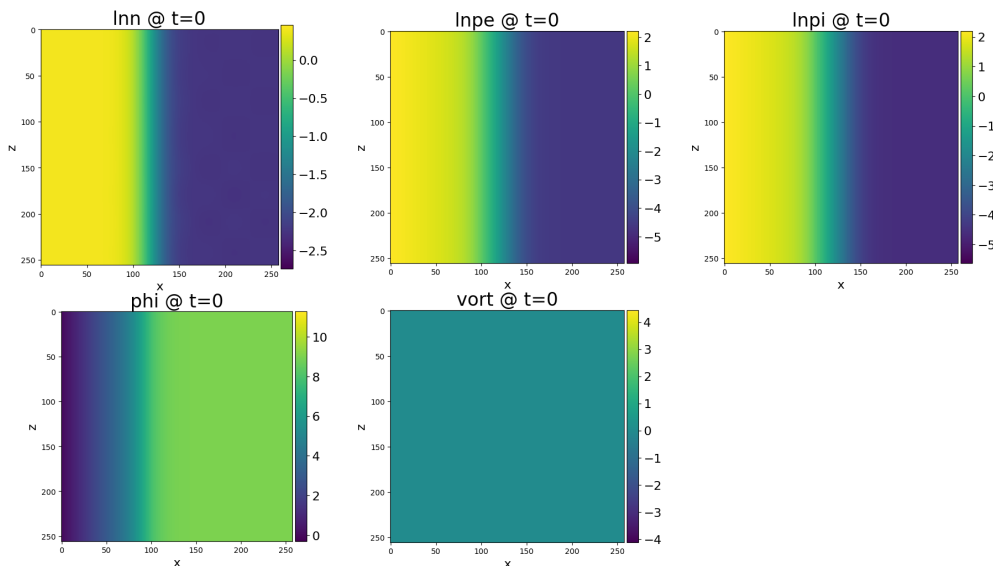


Figure 2: Standard simuleringen `Alexander_pi` til $t = 0$. Akserne er i numeriske grid enheder og ghost cells inkluderet. Fysiske enheder er $z = 140/\rho_s$, $x = 140/\rho_s + ghostcells$

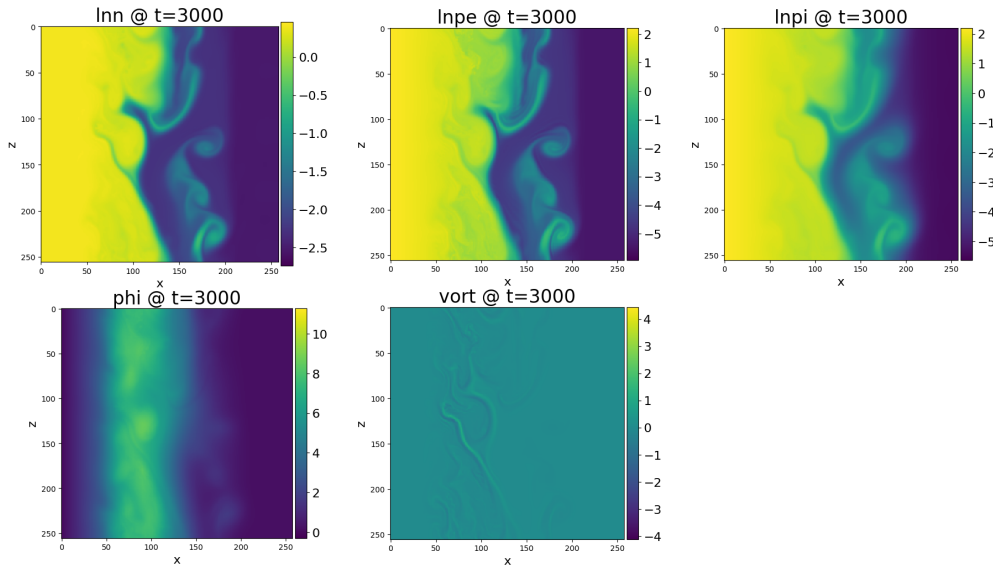


Figure 3: Standard simuleringen **Alexander_pi** til $t = 3000$. Akserne er i numeriske grid enheder og ghost cells inkluderet. Fysiske enheder er $z = 140/\rho_s$, $x = 140/\rho_s + ghostcells$

For tidsudviklingen af feltet $\ln(n)$ for udvalgte simuleringer, se 11. Her blev der udvalgt 2 standard simuleringer og simuleringen af modificeret fysik, og modificeret grid opløsning. Da simuleringerne af modificeret timesteps og den sidste standard simulering har samme fysik og gridopløsning er disse undladt at have med. Ellers kan alle billederne ses i folderen **billeder**. Hvis man har simuleringens data kan man bruge `hesel_scraper/visualizer.ipynb` til se udviklingen af alle felter for én simulering.

4.1.4 Opdeling af data til eksperiment

Formålet med eksperimenterne er at undersøge modellens evne til at forudsige den næste tilstand, iterativt approksimere tidsudviklingsoperatoren, generalisere til både kendte og ukendte fysiske begyndelsesbetingelser samt overføre viden mellem simuleringer med forskellige rumlige opløsninger.

For at muliggøre disse undersøgelser anvendes kun datasættene **Alexander_phi_0** og **Alexander_e** til træning, mens de resterende simuleringer reserveres til test. Dette gør det muligt at evaluere modellens generaliseringsevne på fysiske scenarier, som ikke er observeret under træningen.

Datasættet **Alexander_pi** indeholder en længere tidsserie end både **Alexander_phi_0** og **Alexander_e**. Det anvendes derfor som et særligt testsæt til at undersøge modellens evne til at forudsige plasmaets udvikling over længere tidshorisonter end dem, der indgår i træningsdataene.

Da modellen både skal kunne interpolere og ekstrapolere i tidsdimensionen, opdeles datasættene forskelligt. Tidsrækken i **Alexander_e** opdeles kronologisk, således at de tidligste observationer anvendes til træning, mens senere observationer anvendes til validering og test. For **Alexander_phi_0** foretages derimod en tilfældig opdeling af samples.

For begge datasæt anvendes 70% af data til træning, 15% til validering og 15% til test.

4.1.5 Domæne for modelarkitektur

Domænet for antal lag, aktiveringsfunktioner og hidden dimension blev diskuteret i afsnit 2.1.3. På den baggrund undersøges følgende kombinationer:

$$\text{Antal lag} \in \{2, 4, 6, 8\},$$

$$\text{Aktiveringsfunktion} \in \{\tanh, \text{SiLU}, \text{blanding af } \tanh \text{ og SiLU}\},$$

$$\text{Hidden dimension} \in \{256, 512, 1024, 2048, 4096\}.$$

Først kommer Hidden dimension til at være låst fast til 256. Så det kun er antallet af lag og aktiveringsfunktion der varierer. Herefter vælges nogle af de bedste aktiveringsfunktion og antal lag, og så undersøges det bedste hidden dimension konfiguration.

Da SiLU aktiveringsfunktionen er yderst sensitiv overfor learning rate, kommer learning rate til at variere imellem eksperimenterne. Når der kommer flere lag i netværket, bliver blandingen af aktiveringsfunktioner opdelt i fire grupper, første gruppe har tanh og SiLU skiftevis, anden gruppe har SiLU og tanh skiftevis tredje gruppe har første halvdel tanh og anden halvdel SiLU, fjerde gruppe har første halvdel SiLU og anden halvdel tanh. Dette gøres for at give begge aktiveringsfunktioner mulighed for at konvergere.

5 Resultater og diskution

Dette kapitel præsenterer kun resultaterne af de forskellige tests og målinger, der er blevet udført i løbet af projektet. Her vil resultater også diskuteres i forhold til de forskellige eksperimenter, og hvordan de bruges.

5.1 Udforskning af modelarkitektur og hyperparametre

Der udføres kun en udforskning af modelarkitektur og hyperparametre på PINN_z, da data'en og PDE loss er helt den samme for alle modeltyper. Det eneste der varierer er input og output dimension, hvilket ikke burde have en stor indflydelse på model arkitektur.

Alt data'en for denne udforskning ligger i eksperimentmapperne under `Experimenter/PINN_z`. Opdeling af foldere med eksperiment nummer og trial nummer, er som følgerne:

Eks er eksperiment nummer der referer til folderen `ex_<>_i`, indekset $i \in \{1, 2, 3, 4\}$ angiver specifikationen der er brugt i trial $\{_, _, _, _\}$.

For at få mest mulig information om arkitekturerne blev blandinger af aktiveringsfunktioner valgt således:

$$struktur_{9,\{1,2\}} = \{S, t\} \quad (56)$$

$$struktur_{9,\{3,4\}} = \{t, S\} \quad (57)$$

$$struktur_{10,1} = \{S, t, S, t\} \quad (58)$$

$$struktur_{10,2} = \{t, S, t, S\} \quad (59)$$

$$struktur_{10,3} = \{S, S, t, t\} \quad (60)$$

$$struktur_{10,4} = \{t, t, S, S\} \quad (61)$$

$$struktur_{11,1} = \{S, S, S, t, t, t\} \quad (62)$$

$$struktur_{11,2} = \{t, t, t, S, S, S\} \quad (63)$$

$$struktur_{11,3} = \{S, t, S, t, S, t\} \quad (64)$$

$$struktur_{11,4} = \{t, S, t, S, t, S\} \quad (65)$$

$$struktur_{12,1} = \{S, \dots, S, t, \dots, t\} \quad (66)$$

$$struktur_{12,2} = \{t, \dots, t, S, \dots, S\} \quad (67)$$

$$struktur_{12,3} = \{S, t, S, t, S, t, S, t\} \quad (68)$$

$$struktur_{12,4} = \{t, S, t, S, t, S, t, S\} \quad (69)$$

Her betyder "S" SiLU aktiverings funktion og "t" betyder tanh aktiverings funktion.

Hyperparametrene for denne udforskning af PINN_z var følgende: hidden dimension 256, batch size 512, og 5 epochs.

Der ønskes kun at sortere groft så learning rate blev valgt efter behov så modellerne kunn konvegere noghelunde. For trænings, validering og test kurverne af de forskellige eksperimenter se appendix 12, 13 og 14. Bemærk dette er kun total loss, data og PDE loss er ikke inkluderet da dette ville gøre plotsne svære at læse. Der er blevet brugt moving average med vindue størrelse på 3. For plottet 13 er der loss i træning og validering som er

ekstremt højt, dette er grundet den voldsomme ustabilitet af træning af SiLU netværk med relativt høj learning rate. For validering og test loss bliver tidsrække og random samplet datasættene kørt hvert for sig, hvilket giver de spring i validering og test kurverne. I hver epoch valideres der først på random sample split, hvilket viser interpolations evne, og derefter på tidsrække split, hvilket viser ekstrapolations evne. Dette gør det selvfølgelig nemt at se forskellen mellem inter og ekstrapolation, men svært at se om der er en klar tendens i validering. Dette håndteres i early stopping ved at tage samlet gennemsnit af validering.

Avg test loss	Test loss	Lr	Funk	Lag	Eks
1.70	{1.9, 1.2, 1.9, 1.8}	$\{5, \dots, 5\} \cdot 10^{(-5)}$	$\{\{s, s, s, s\}, \dots\}$	4	6
2.00	{1.9, 2.3, 2.0, 1.8}	$\{5, \dots, 5\} \cdot 10^{(-5)}$	$\{\{s, s\}, \dots\}$	2	5
2.20	{1.9, Na, 2.4, 2.3}	$\{5, 5, 1, 1\} \cdot 10^{(-5)}$	$\{\{s, s, s, s, s, s, s, s\}, \dots\}$	8	8
2.40	{Na, Na, 2.4, 2.4}	$\{5, \dots, 5\} \cdot 10^{(-5)}$	$\{\{s, s, s, s, s, s\}, \dots\}$	6	7
3.38	{4.3, 2.4, 4.3, 2.5}	$\{1, 1, 1, 1\} \cdot 10^{(-3)}$	{58, 59, 60, 61}	4	10
3.98	{5.0, 3.5, 4.2, 3.2}	$\{1, 1, 1, 0.1\} \cdot 10^{(-3)}$	{66, 67, 68, 69}	8	12
4.35	{4.5, 4.4, 4.2, 4.3}	$\{1, 1, 1, 1\} \cdot 10^{(-3)}$	$\{\{t, t\}, \dots\}$	2	1
4.60	{4.6, 7.7, 3.5, 2.6}	$\{1, 1, 1, 1\} \cdot 10^{(-3)}$	{56, 56, 57, 57}	2	9
4.95	{5.1, 4.6, 5.1, 5.0}	$\{1, 1, 1, 1\} \cdot 10^{(-3)}$	$\{\{t, t, t, t, t, t\}, \dots\}$	6	3
5.03	{5.0, 5.0, 5.1, 5.0}	$\{1, 1, 1, 1\} \cdot 10^{(-3)}$	$\{\{t, t, t, t\}, \dots\}$	4	2
5.08	{5.1, 5.1, 5.0, 5.1}	$\{1, 1, 1, 1\} \cdot 10^{(-3)}$	$\{\{t, t, t, t, t, t, t, t\}, \dots\}$	8	4
5.33	{5.8, 5.0, 5.2, Na}	$\{1, 1, 1, 0.1\} \cdot 10^{(-3)}$	{62, 63, 64, 65}	6	11

Table 2: PINN_z strukturer sorteret efter bedste **Avg test loss**

Her er **Avg test loss** er gennemsnitlige traial test loss. **Test loss** er de enkelte trial gennemsnitlige test loss, **Lr** er learning rate, **Funk** er aktiverings funktioner i de forskellige lag, **Lag** er antal skjulte lag.

Bemærk der hvor der står Na, er det fordi træningen kollapsede og gav ikke endelige værdier. De ikke endelige værdier var udelukkende fra PDE loss. Da det ikke vurderes at give nyttig information at køre disse igennem til test blev de bare stoppet.

Valget af aktiverings funktion bestemmes til SiLU, og antal lag til 4, da disse modeller havde klart bedre **Avg test loss**. Det kan dog bemærkes at eksperiment 10 og 12 trail 2, og 4, der var rimelige ens modeller, havde relativt lav loss, men dog ikke noget der nærmer sig SiLU modellerne. Dette peger samlet på, at aktiveringsfunktionen havde større betydning end små variationer i lagopbygningen i denne første udforskning.

Resultaterne af varierende hidden dimension kan ses i tabel 3. Her bestemmes learning rate igen så modellerne træner stabil og stadig kan konvergere nogenlunde. Hyper parametre er som følgende: batch size 1024, 4 lag og 10 epochs.

Der blev kun kørt én trial for hver hidden dimension.

Hertil skal det noteres at eksperiment 15 med hidden dimension 4096 havde bedste validerings loss i epoch 5, og derefter steg validerings lossen igen, hvilket kan være tegn på overfitting. Samtidig tyder tabellen på, at en mellemstor model gav den bedste balance mellem kapacitet og stabil generalisering.

Test loss	Hidden dim	Lr	Eks
1.64	1024	$3 \cdot 10^{-5}$	13
1.75	256	$5 \cdot 10^{-5}$	14
2.03	4096	$1 \cdot 10^{-5}$	15

Table 3: Resultater af udforskning med varierende hidden dimension

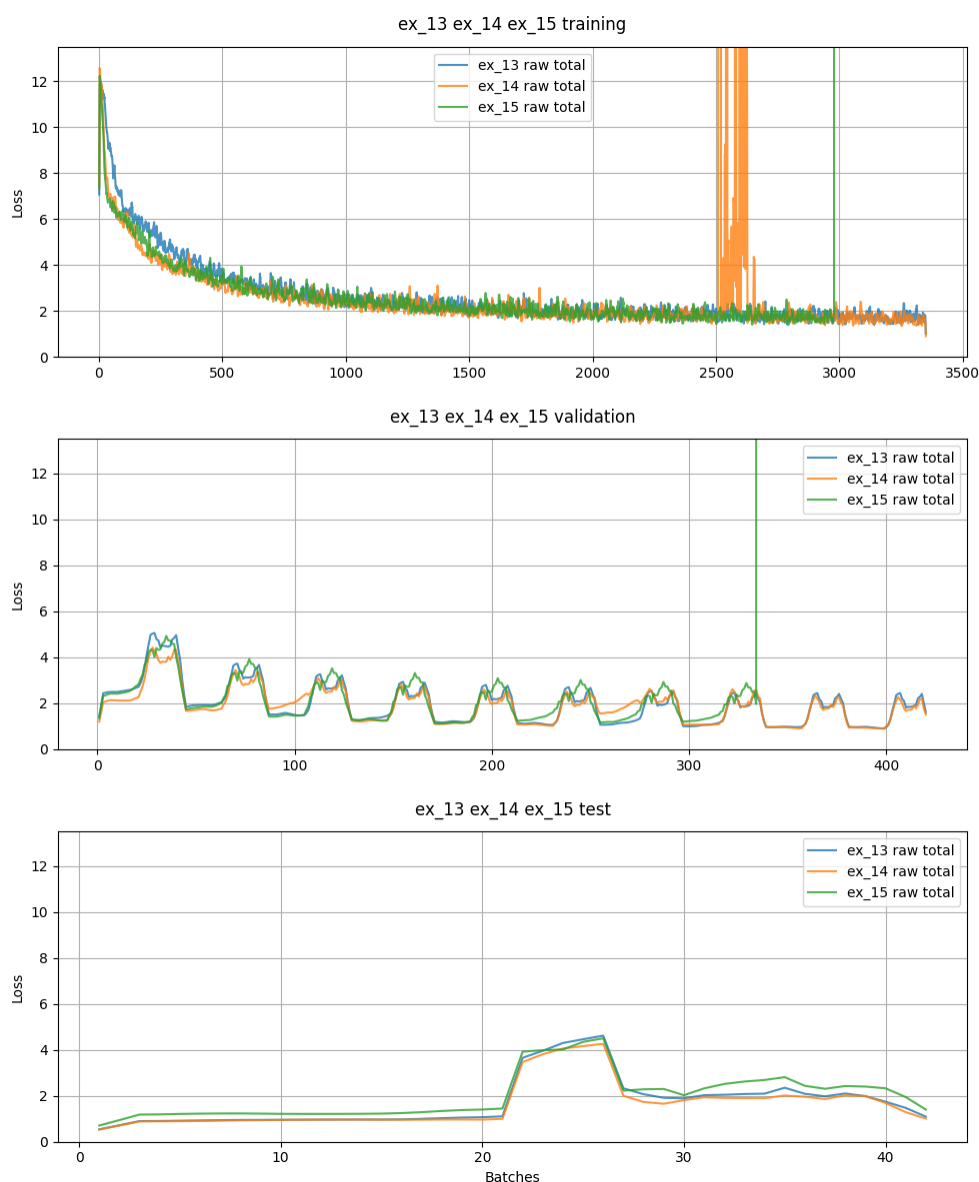


Figure 4: Trænings-, validerings- og testkurver for eksperiment 13 med varierende hidden dimension

Test loss uvægtet	Lag	Hidden dimension	Lr	eks
1.73	2	8, 8	$1 \cdot 10^{-4}$	3
1.97	3	4, 4, 4	$7 \cdot 10^{-4}$	7
1.99	3	16, 16, 16	$3 \cdot 10^{-4}$	9
2.02	3	8, 8, 8	$3 \cdot 10^{-4}$	8
2.04	2	16, 16	$1 \cdot 10^{-4}$	4
2.19	2	4, 4	$1 \cdot 10^{-3}$	2
2.19	2	32, 32	$5 \cdot 10^{-5}$	5
4.51	2	2, 2	$1 \cdot 10^{-3}$	1
4.95	3	2, 2, 2	$1 \cdot 10^{-3}$	6

Table 4: Resultater af hyperparameterudforskning med varierende hidden dimension sorteret efter uvægtet test loss

Til endelige eksperiment vælges modellen med hidden dimension 1024 og 4 lag. En vigtig note er at alle foregående eksperimenter konvergerede PDE loss mod omkring 10^{-5} til 10^{-4} og data loss mod omkring 2 til 10.

5.1.1 Model arkitektur for endelige eksperiment

Desværre overfittede de større modeller og da mindre modeller blev testet viste det sig at alle disse modeller overfittede. Derfor var det eneste man kunne tage med fra udforskningen i kapitel 5.1 var, at tanh som aktiveringsfunktion var værre end SiLU, data loss konvergerede mod 2, PDE loss konvergerede mod 10^{-4} , og PDE loss var meget ustabil. Da konvergenzen af data loss og PDE loss kunne approksimeres skales data og PDE loss med deres approksimerede konvergensværdier. Derved sættes loss til:

$$loss_{total} = 2 \cdot data_{loss} + 10^{-4} \cdot pde_{loss} \quad (70)$$

Da PDE loss skales med samme skala som data loss, medfører det at PDE loss er meget mere stabil og learning rate kan komme højere op. Dette kommer af at når modellen skal afveje PDE loss mod data loss bliver den straffet hårdt med det samme. Dog blev det opsaveret at hvis modellen fik kæmpe loss i første batches, så kollapsede træningen og gav NaN værdier og kunne ikke redes. Learning rate blev sat så højt som muligt uden at PDE loss blev ustabil, så modellerne kunne konvergere så hurtigt som muligt. Hyperparametrene blev sat til: batch størrelse på 1024, træning, validering og test split på [0, 80%], (80%, 90%) og (90%, 100%]. Hvor `Alexander_e` splittes med tidsrække og `Alexander_phi_0` splittes med random sample, 100 epochs, og early stopping patience på 10 og min_delta på 0.1.

Alt data findes i eksperimentmapperne under `Experimenter/PINN_z`.

Her skal det bemærkes at selvom det normal medfører øget ustabilitet at have større model, vidste det sig at modellerne i eksperiment 8, 9 var mere stabile end eksperiment 6 og 7 og derfor havde højere learning rate. Alle modeller konvergerede inden de nåede 100 epochs se fig 15 og fig 16. Her skal det også bemærkes at modellerne i eksperiment 2 og 6 nåede til omkring epoch 95 før early stopping blev aktiveret. Det vigtigste resultat her er,

at de bedste testtab opnås med små modeller, hvilket tyder på at større modeller ikke gav ekstra gevinst på dette datasæt.

Arkitekturen for den endelige model bliver derfor bestemt til 2 lag med 8 hidden dimensioner.

5.2 Endelige eksperiment

5.2.1 Endelige eksperiment for PINN_z

Den endelige trænedte model for PINN_z er bestemt ud fra resultaterne af udforskningen af modelarkitektur og hyperparametre i kapitel 5.1. Her er hyperparametrene sat til at være: learning rate 10^{-4} , hidden dimension (8, 8), aktiveringsfunktion er SiLU, antal lag 2, data loss vægtes med 0.25, PDE loss vægtes med 10^4 , input er hele x rækken, med en z bredde på 1 nabe celle i begge rætninger, batch størrelse på 1024, træning, validering og test split på [0, 80%], (80%, 90%) og (90%, 100%], hvor **Alexander_e** splittes med tidsrække og **Alexander_phi_0** splittes med random sample, 200 epochs, og early stopping patience på 30 og min_delta på 0.0. Denne opsætning er valgt, fordi den i den forrige udforskning gav den bedste kombination af lav test loss og stabil træning.

Der blev kørt 3 eksperimenter med denne arkitektur hvilket gav følgende træning, validering og test kurver:

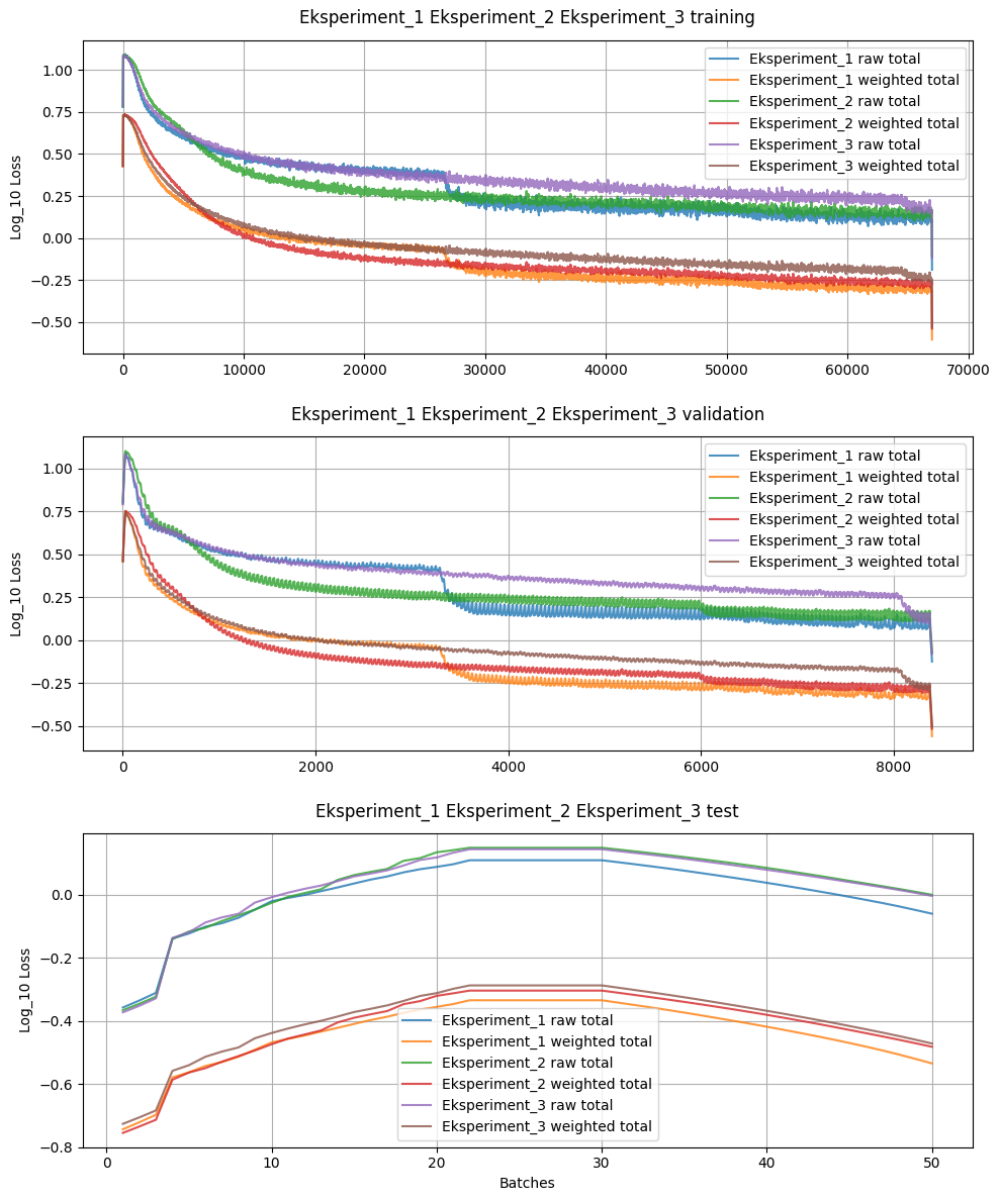


Figure 5: Endelige eksperimenter for PINN_z med 3 forskellige random seeds, vægtet og uvægtet loss, og moving average med vindue størrelse 50.

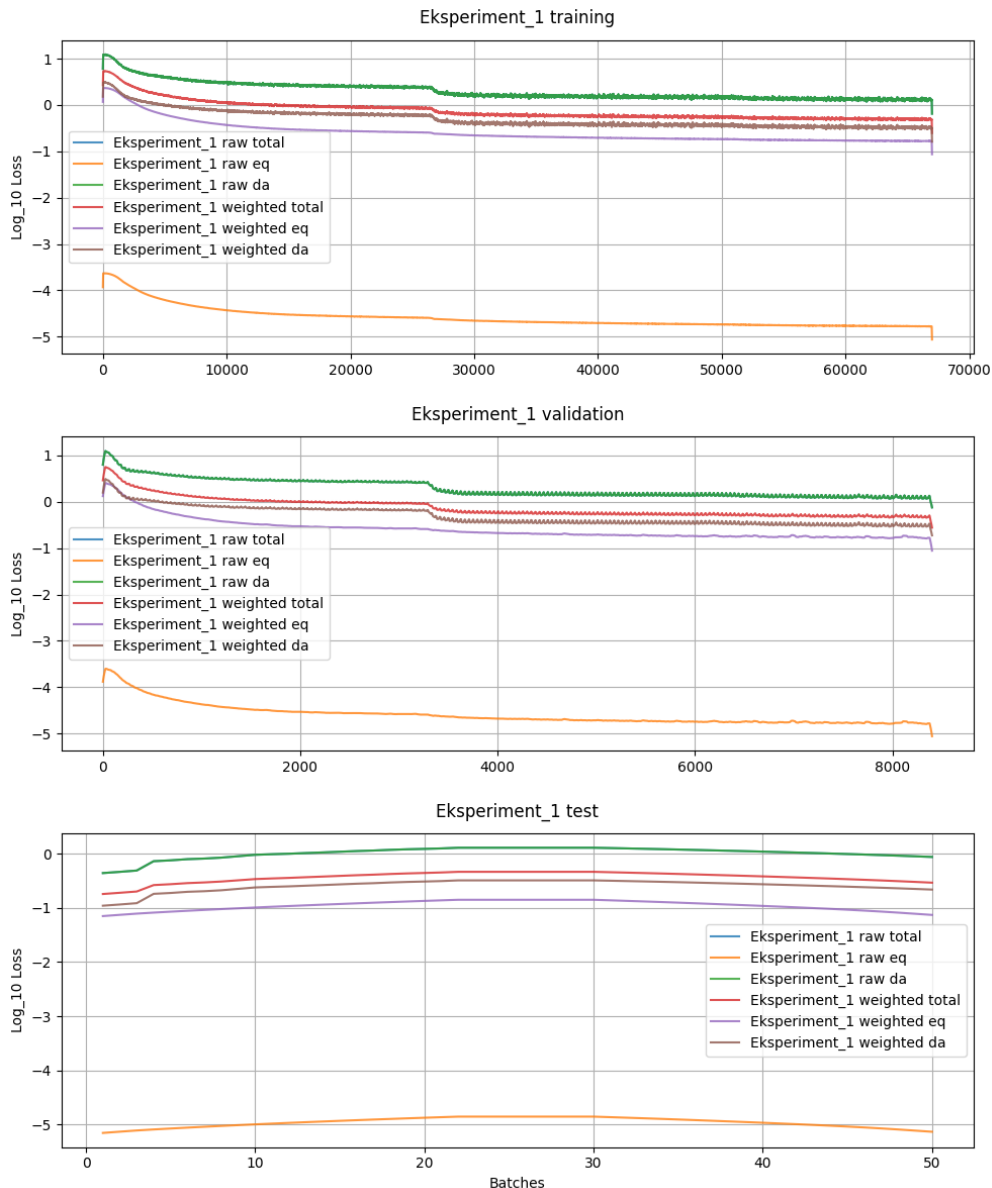


Figure 6: Endelige eksperiment for PINN_z, Eksperiment 1, med vægtede og uvægtede losses for total, data og PDE (eq), samt moving average med vindue størrelse 50.

På figur 5 ses at alle 3 eksperimenter har en meget lignende udvikling og konvergere mod samme resultat. På figur 6 ses det ikke tydeligt men for uvægtede loss, ligger data og total loss næsten oven i hinanden, mens PDE (eq) ligger helt i bunden. For de vægtede loss, ligger data og PDE (eq) loss næsten oven i hinanden, hvilket betyder de vægtes næsten lige meget og giver meget stabil træning. Her til skal det siges at ingen af modellerne konvergerede helt inden for de 200 epochs. Det tyder på, at den valgte vægtning gjorde optimeringen mere robust uden helt at fjerne behovet for længere træning.

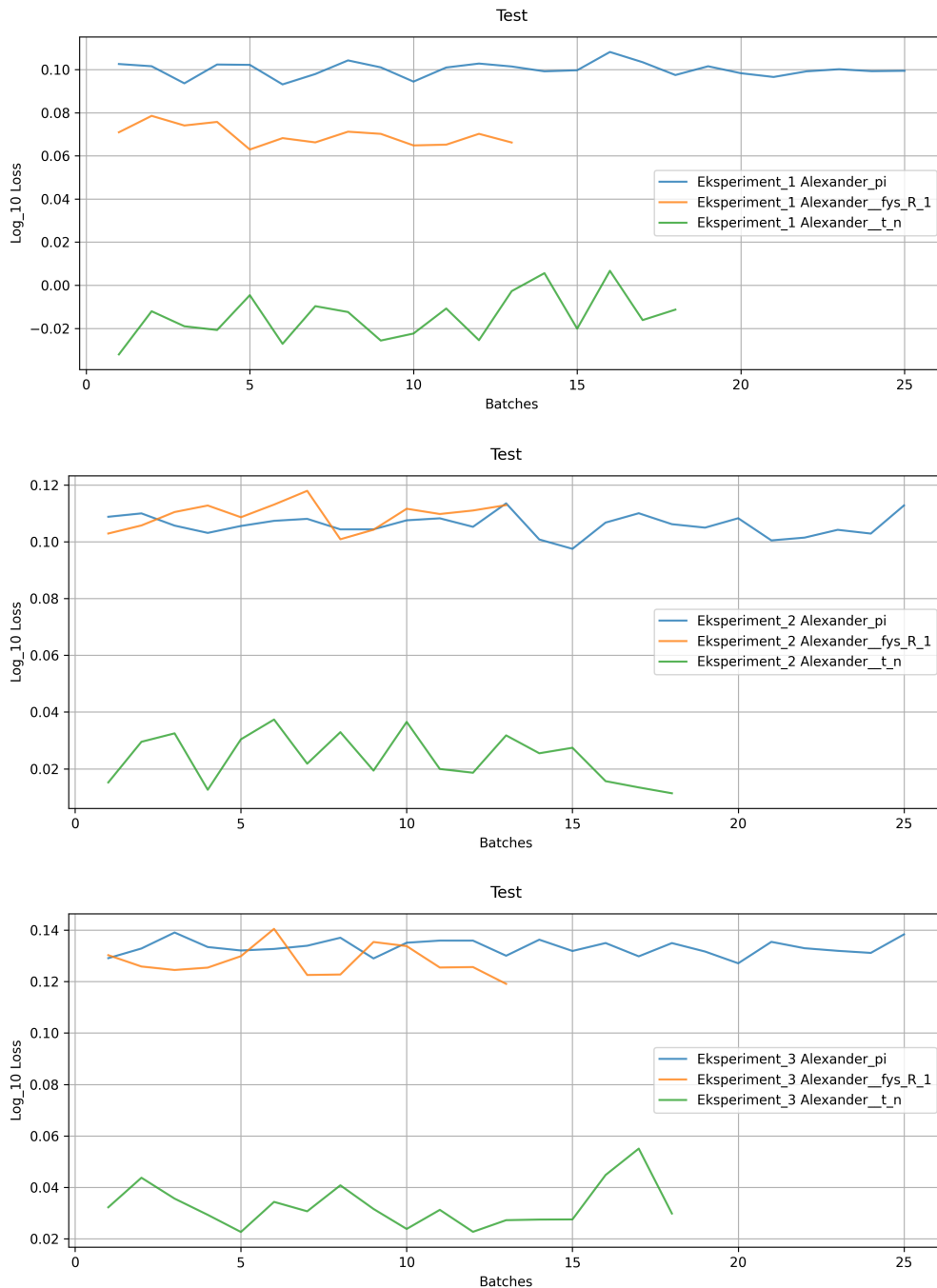


Figure 7: Total uvægtet loss på testdatasættene for Eksperiment 1, 2 og 3, alle med batch-størrelse 1024.

På figur 7, ses det at alle 3 eksperimenter har ens test kurver for alle ukendte datasæt. Alle modeller sig bedst på datasættet med længere tidsstep, frem for kendt og ukend fysik. Dette kan være tegn på overfitting af data. Da dette er uvægtet loss, har PDE loss også en meget lav loss. Det samlede billede er derfor, at modellen generaliserer ens på tværs af seeds, men stadig har sværere ved ændret fysik end ved ændret tidsdiskretisering.

Det endelige billede ser nogenlunde således ud:

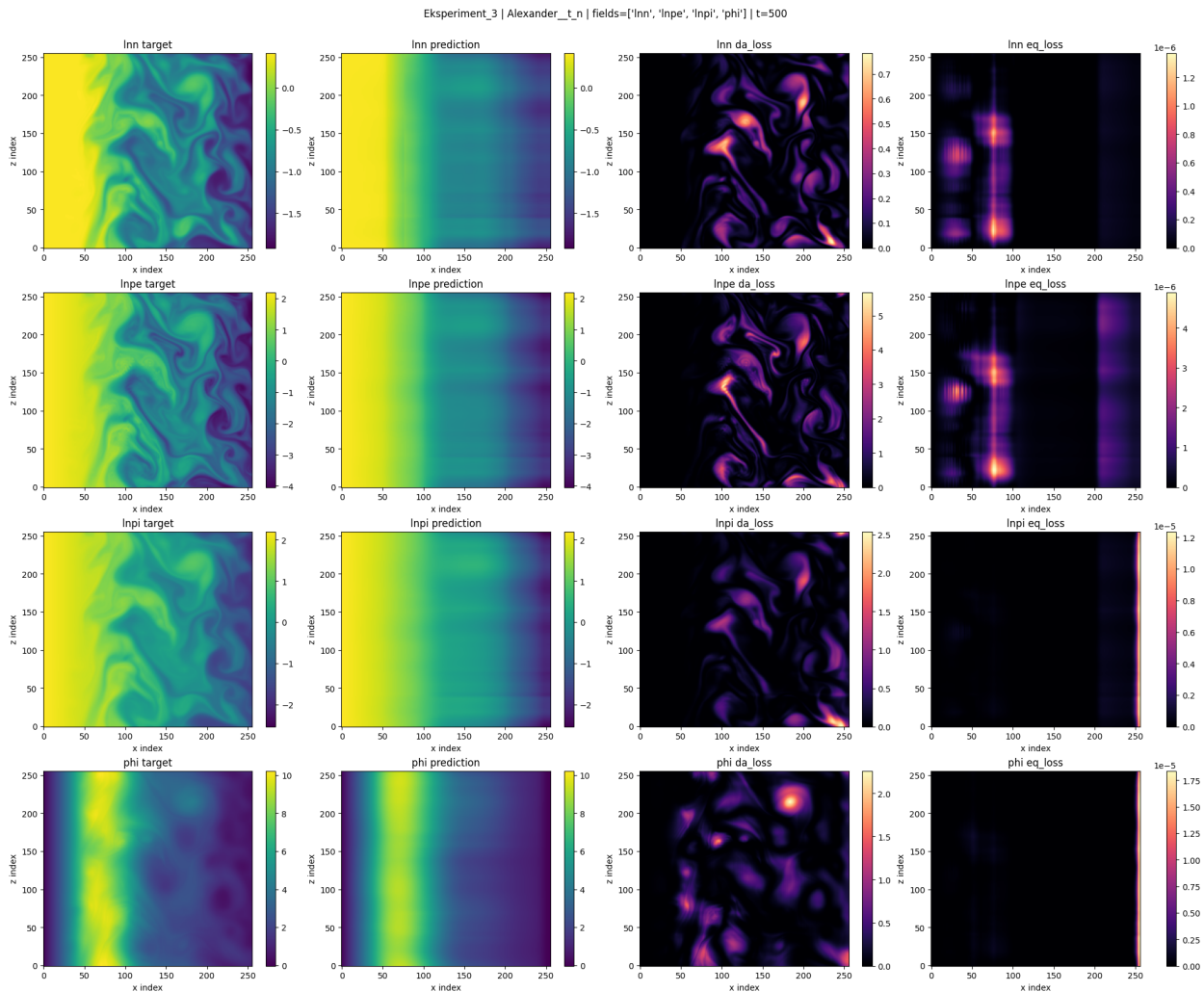


Figure 8: Modellens mål at forudsige `Alexander__t_n` ved `t_idx = 500` og `t_fys = 12500`, sammenlignet med det faktiske resultat, data loss og PDE (eq) loss.

Selvom modellen ikke rammer dynamikkerne i data'en serlig godt, rammer den fysikken til yderst højt nøjagtighed da modellen vægter PDE (eq) loss ekstremt højt. Deraf burde det være muligt at ofre PDE (eq) loss og træningsstabilitet for data nøjagtighed.

5.2.2 Endelige eksperiment for PINN

Da PINN havde problemer med træning sættes model arkitekturen tils at være: For lag 1 var hidden dimension 256 og aktiveringsfunktion SiLU, for lag 2 var hidden dimension 128 og aktiveringsfunktion SiLU, for lag 3 var hidden dimension 128 og aktiveringsfunktion SiLU, for lag 4 var hidden dimension 128 og aktiveringsfunktion Tanh, for lag 5 var hidden dimension 64 og aktiveringsfunktion SiLU. Hyper parametrene var for denne model: Træningsdatasæt var `Alexander_e`, med tidsække split med 80% træning 10%validering og 10%test, 1 epoch,

learning rate på 10^{-4} , batch størrelse på 4000, data loss vægtet med 500 og PDE (eq) loss vægtet med 10^3 , og initial og rand betingelser vægtet 1. Dette gav trænings, validerings og test kurver som ses på figur 9.

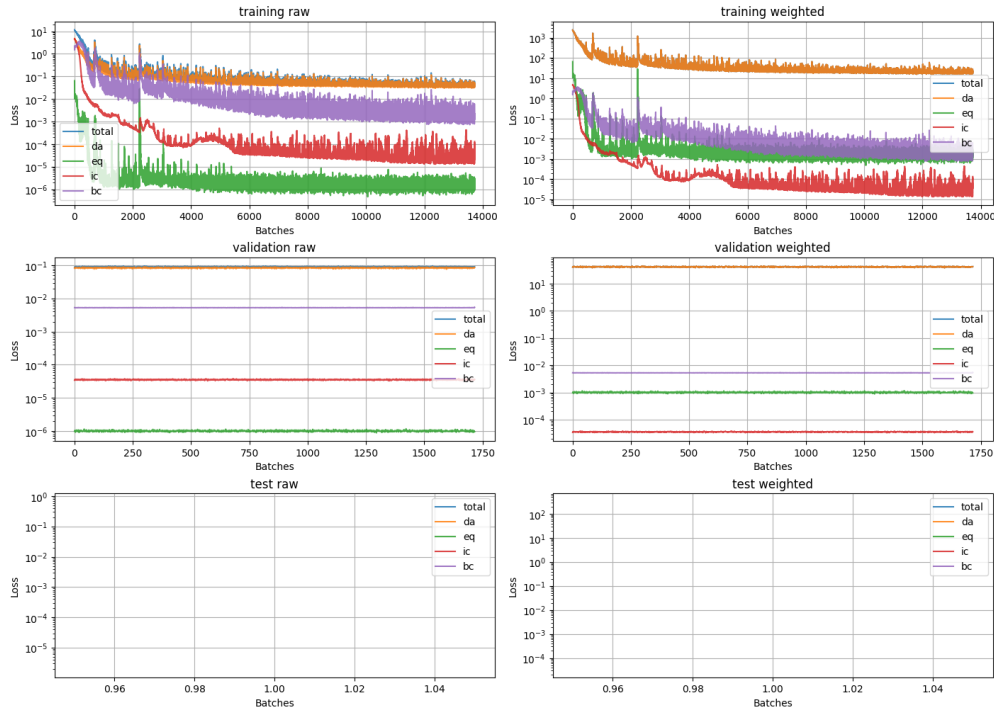


Figure 9: Endelige eksperimenter for PINN, vægtet og uvægtet loss

Her var `test_run` den endelige model, der blev trænet og testet. Den havde en gennemsnitlig uvægtet test loss på de ukendte datasæt var som følgerne:

Datasæt	Total loss	Data loss	PDE (eq) loss	IC loss	BC loss
Alexander_pi	$1.531 \cdot 10^{-1}$	$1.105 \cdot 10^{-1}$	$5.436 \cdot 10^{-6}$	$2.575 \cdot 10^{-2}$	$1.680 \cdot 10^{-2}$
Alexander__fys_R_q	$6.785 \cdot 10^{-1}$	$2.454 \cdot 10^{-1}$	$1.687 \cdot 10^{-6}$	$4.306 \cdot 10^{-1}$	$2.549 \cdot 10^{-3}$
Alexander__t_n	$5.796 \cdot 10^{-1}$	$5.441 \cdot 10^{-1}$	$1.237 \cdot 10^{-5}$	$1.373 \cdot 10^{-3}$	$3.410 \cdot 10^{-2}$
Alexander__grid_c_0	$6.462 \cdot 10^{-1}$	$4.723 \cdot 10^{-2}$	$2.556 \cdot 10^{-6}$	$5.955 \cdot 10^{-1}$	$3.420 \cdot 10^{-3}$

Table 5: Uvægtede test losses for den endelige PINN model på de ukendte datasæt, opdelt i total loss, data loss, PDE (eq) loss, IC loss og BC loss.

For den endelige visualisering af model prediktion bruges grid da den har bedste loss.

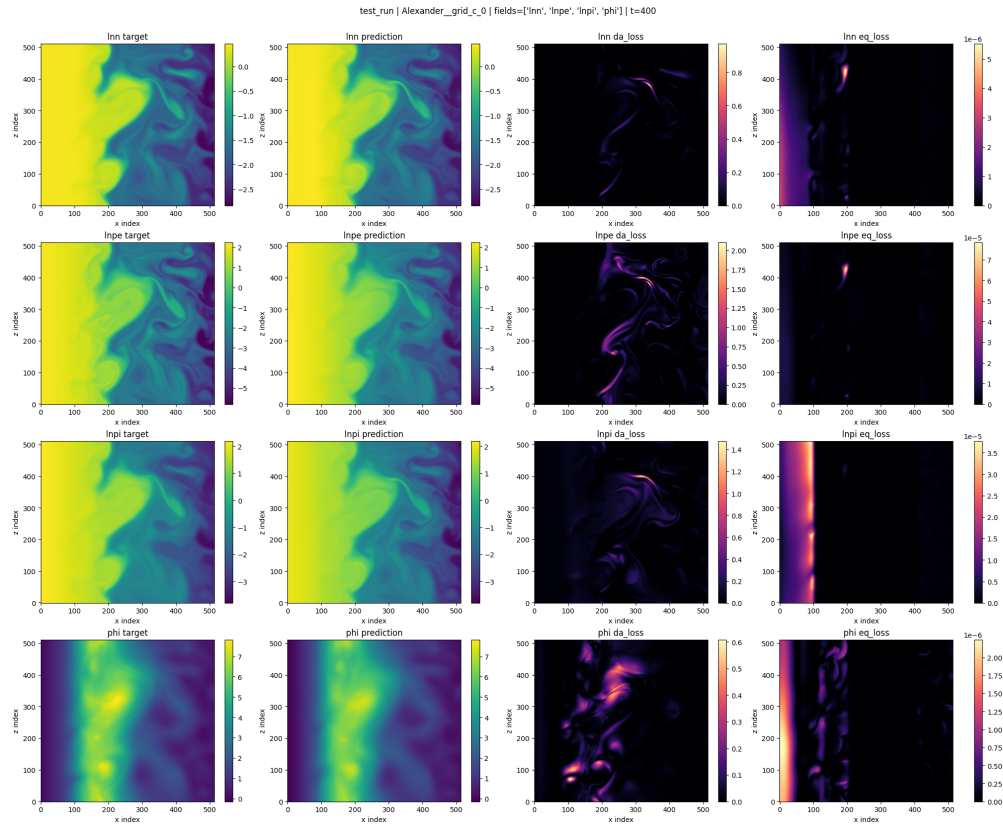


Figure 10: Modellens mål at forudsige Alexander_grid_c_0 ved $t_{\text{idx}} = 400$ og $t_{\text{fys}} = 00$, sammenlignet med det faktiske resultat, data loss, PDE (eq) loss

Det ses på figur 10 at modellen rammer meget bedre data dynamikkerne end hvad PINN_z modellen gjorde.

6 Konklusion

Projektet viser, at Scientific Machine Learning (SciML) har potentiale til at blive anvendt til modellering af randdynamik i plasma beskrevet ved HESEL-modellen. Resultaterne indikerer dog, at de undersøgte modeller endnu ikke er modne nok til at erstatte klassiske numeriske simuleringer. Til gengæld viser de lovende egenskaber som approksimationer af tidsudviklingsoperatoren, hvor de kan forudsige næste tilstand med betydeligt lavere beregningsomkostninger end en fuld numerisk løsning.

Særligt blev det vist, at simple fysikinformede modeller kan opnå høj nøjagtighed ved forudsigelse af lokale dynamikker, selv når de kun anvender information fra et begrænset område af domænet. Dette peger på, at fysikinformeret maskinlæring kan fungere som et nyttigt supplement til eksisterende simuleringsværktøjer.

Forskningsspørgsmålene kan på denne baggrund besvares som følger:

1. Hvordan kan fysikinformede neurale netværk anvendes til at modellere HESEL-modellen?

Fysikinformede neurale netværk kan anvendes til at lære plasmaets tidsudvikling ved at kombinere simuleringsdata med de underliggende differentialligninger. Resultaterne viser, at modellerne endnu ikke kan erstatte klassiske simuleringer som selvstændig metode, da præcisionen fortsat er utilstrækkelig til dette formål. De udviser dog lovende egenskaber som supplement til numeriske metoder, særligt når målet er hurtig approksimation af næste tilstand.

2. Hvilken betydning har vægtningen mellem data-loss og PDE-loss, og hvordan påvirker dataopløsningen resultaterne?

Vægtningen mellem data-loss og PDE-loss har stor betydning for modellernes adfærd. PDE-loss bidrog i flere tilfælde til ustabil træning, men forbedrede samtidig modellernes evne til at generalisere på tværs af forskellige dataopløsninger og ukendte simuleringskonfigurationer. Resultaterne peger derfor på et vigtigt kompromis mellem træningsstabilitet og fysisk konsistens.

3. I hvor høj grad kan modellen generalisere til ukendte punkter, tidsskridt og simulationskonfigurationer?

Modellerne demonstrerede en tilfredsstillende generaliseringsevne på tværs af både ukendte tidspunkter og ændrede simulationskonfigurationer. Resultaterne tyder på, at den fysiske information indlejret i træningen hjælper modellerne med at bevare meningsfulde forudsigelser, selv når de anvendes uden for træningsdatasættet.

4. Hvor meget kan modellen effektiviseres, før tabet i præcision bliver væsentligt?

Resultaterne viser, at modelkompleksiteten har stor betydning for træningsforløbet. For PINN_z-modellen var simple arkitekturer generelt mere stabile end større modeller, som ofte førte til ustabiliteter i PDE-loss. Omvendt krævede den punktbaserede

PINN-model en relativt stor modelkapacitet for overhovedet at kunne lære de observerede dynamikker. Der findes derfor ikke én optimal kompleksitet, men snarere et kompromis mellem stabilitet, modelkapacitet og præcision.

Den samlede konklusion er, at fysikinformede neurale netværk udgør et lovende supplement til klassiske numeriske metoder inden for simulering af randplasma. Modellerne kan endnu ikke erstatte traditionelle simuleringer, men de demonstrerer en evne til at lære væsentlige dele af plasmaets dynamik og til at generalisere til nye scenarier.

Resultaterne viser samtidig, at størrelsen af det område, modellen skal forudsige, har stor betydning for træningsstabiliteten. Forudsigelse af større dele af domænet medførte markant vanskeligere optimeringsproblemer og større risiko for ustabile træningsforløb.

Endelig peger resultaterne på, at fysikinformede modeller kan udnytte viden fra stabile simulationskonfigurationer til at foretage forudsigelser på mere udfordrende konfigurationer. Dette gør dem interessante som fremtidige surrogate-modeller, der potentielt kan reducere beregningstiden ved numeriske plasmaeksperimenter og understøtte udforskningen af nye parameterum.

List of Figures

1	Påtvungne profiler i numerisk domæne, med ghost cells, "init" er n_p , $p_{e,p}$ og $p_{i,p}$, med sigma profiler der bestemmer hvor Λ termene håndhæves. x-aksen har Bohm normaliseret længde: $x = 140/\rho_s$	12
2	Standard simuleringen Alexander_pi til $t = 0$. Akserne er i numeriske grid enheder og ghost cells inkluderet. Fysiske enheder er $z = 140/\rho_s$, $x = 140/\rho_s + ghostcells$	21
3	Standard simuleringen Alexander_pi til $t = 3000$. Akserne er i numeriske grid enheder og ghost cells inkluderet. Fysiske enheder er $z = 140/\rho_s$, $x = 140/\rho_s + ghostcells$	22
4	Trænings-, validerings- og testkurver for eksperiment 13 med varierende hid-den dimension	26
5	Endelige eksperimenter for PINN_z med 3 forskellige random seeds, vægtet og uvægtet loss, og moving average med vindue størrelse 50.	29
6	Endelige eksperiment for PINN_z, Eksperiment 1, med vægtede og uvægtede losses for total, data og PDE (eq), samt moving average med vindue størrelse 50.	30
7	Total uvægtet loss på testdatasættene for Eksperiment 1, 2 og 3, alle med batchstørrelse 1024.	31
8	Modellens mål at forudsige Alexander__t_n ved $t_idx = 500$ og $t_fys = 12500$, sammenlignet med det faktiske resultat, data loss og PDE (eq) loss. .	32
9	Endelige eksperimenter for PINN, vægtet og uvægtet loss	33
10	Modellens mål at forudsige Alexander__grid_c_0 ved $t_idx = 400$ og $t_fys = 00$, sammenlignet med det faktiske resultat, data loss, PDE (eq) loss . . .	34
11	Tidsudviklingen af feltet $\ln(n)$ for udvalgte simuleringer	V
12	PINN_z overlap for tanh-modeller, med moving average med vindue størrelse på 3.	VI
13	PINN_z overlap for SiLU-modeller, med moving average med vindue størrelse på 3.	VII
14	PINN_z overlap for blandede SiLU- og tanh-modeller, med moving average med vindue størrelse på 3.	VIII
15	PINN_z modeller med 2 lag, med moving average med vindue størrelse på 5. .	IX
16	PINN_z modeller med 3 lag, med moving average med vindue størrelse på 5. .	X

List of Tables

1	Længden af simuleringerne i Bohm normaliserede tidsenheder og antal tidsindekser.	20
2	PINN_z strukturer sorteret efter bedste Avg test loss	25
3	Resultater af udforskning med varierende hidden dimension	26
4	Resultater af hyperparameterudforskning med varierende hidden dimension sorteret efter uvægtet test loss	27
5	Uvægtede test losses for den endelige PINN model på de ukendte datasæt, opdelt i total loss, data loss, PDE (eq) loss, IC loss og BC loss.	33

Nomenklatur

BTE Boltzman transport ligning

DTU Danmarks Tekniske Universitet

HESEL Hot Edge SOL Electrostatic

HPC High Performance Computing

PDE Partielle Differential Ligninger

PINN Physics Informed Neural Networks

SciML Scientific Machine Learning

SOL Scrape off layer

References

- [1] ITER Organization, “The way to new energy,” 2019. Accessed: 2026-05-11.
- [2] A. S. Thrysøe, M. Løiten, J. Madsen, V. Naulin, A. H. Nielsen, and J. J. Rasmussen, “Plasma particle sources due to interactions with neutrals in a turbulent scrape-off layer of a toroidally confined plasma,” *Physics of Plasmas*, vol. 25, p. 032307, 03 2018.
- [3] A. Nielsen, J. Rasmussen, J. Madsen, G. Xu, V. Naulin, J. Olsen, M. Magnussen, S. Hansen, N. Yan, L. Tophøj, and B. Wan, “Numerical simulations of blobs with ion dynamics,” *Plasma Physics and Controlled Fusion*, vol. 59, no. 2, 2017.
- [4] A. Thrysøe, M. Løiten, J. Madsen, V. Naulin, A. Nielsen, and J. Rasmussen, “Plasma particle sources due to interactions with neutrals in a turbulent scrape-off layer of a toroidally confined plasma,” *Physics of Plasmas*, vol. 25, no. 3, 2018.
- [5] I. E. Lagaris, A. Likas, and D. I. Fotiadis, “Artificial neural networks for solving ordinary and partial differential equations,” *IEEE Transactions on Neural Networks*, vol. 9, no. 5, pp. 987–1000, 1998.
- [6] M. Raissi, P. Perdikaris, and G. E. Karniadakis, “Physics informed deep learning (part i): Data-driven solutions of nonlinear partial differential equations,” *arXiv preprint arXiv:1711.10561*, 2017.
- [7] S. I. Braginskii, “Transport processes in a plasma,” *Reviews of Plasma Physics*, 1, 205., vol. 1, pp. 205–311, 1965. Accessed: 2026-06-11.
- [8] A. Antoft, “Bachelorprojekt: Physics informed neural networks and operators for hesel simulations.” <https://github.com/bantoft/Bachelorprojekt>, 2026. Mit github repository for dette projekt.
- [9] B. Dudson, M. Umansky, X. Xu, P. Snyder, and H. Wilson, “Bout++: A framework for parallel plasma fluid simulations,” *Computer Physics Communications*, vol. 180, pp. 1467–1480, Sep 2009.

7 Appendix A

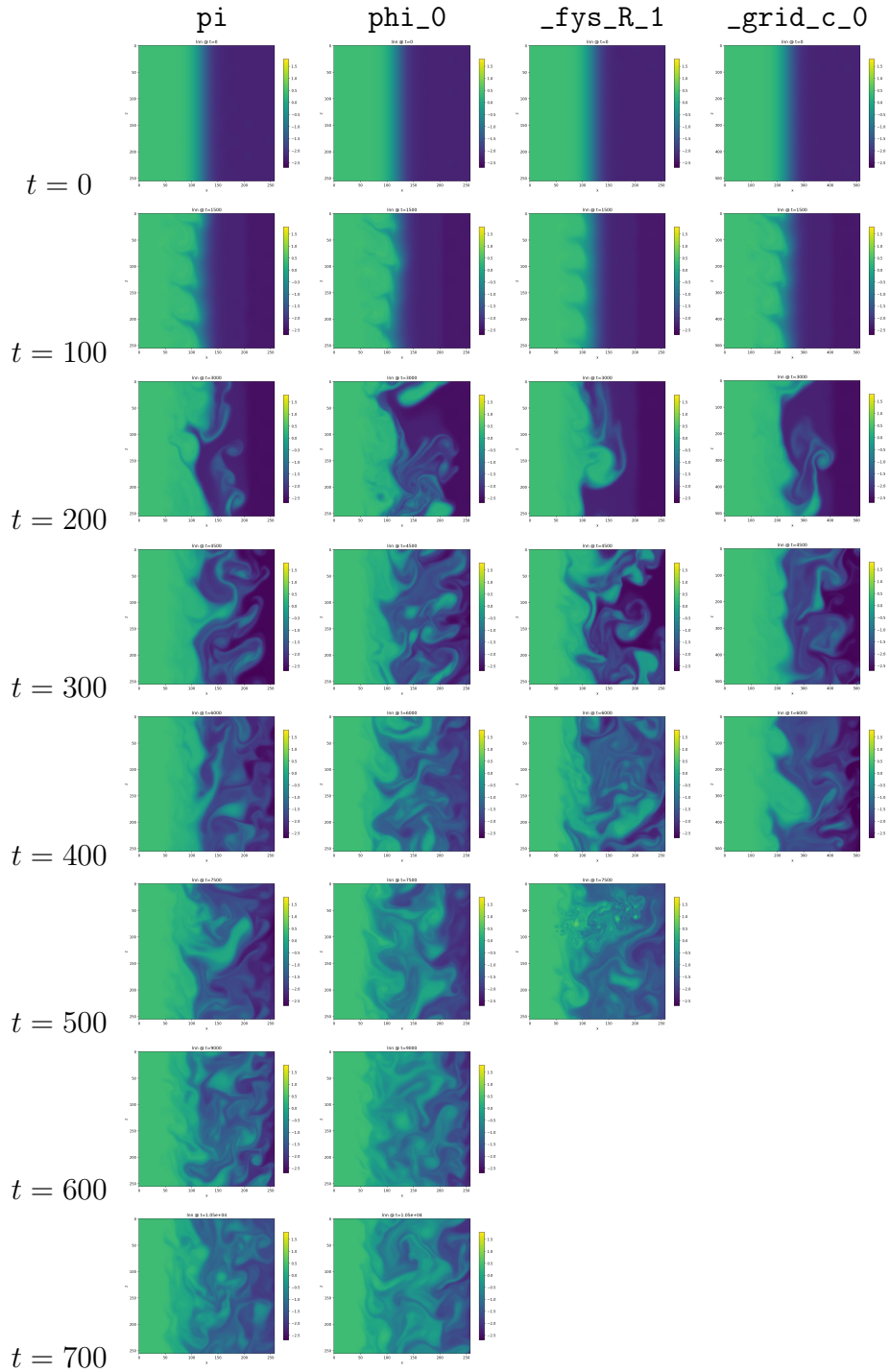


Figure 11: Tidsudviklingen af feltet $\ln(n)$ for udvalgte simuleringer

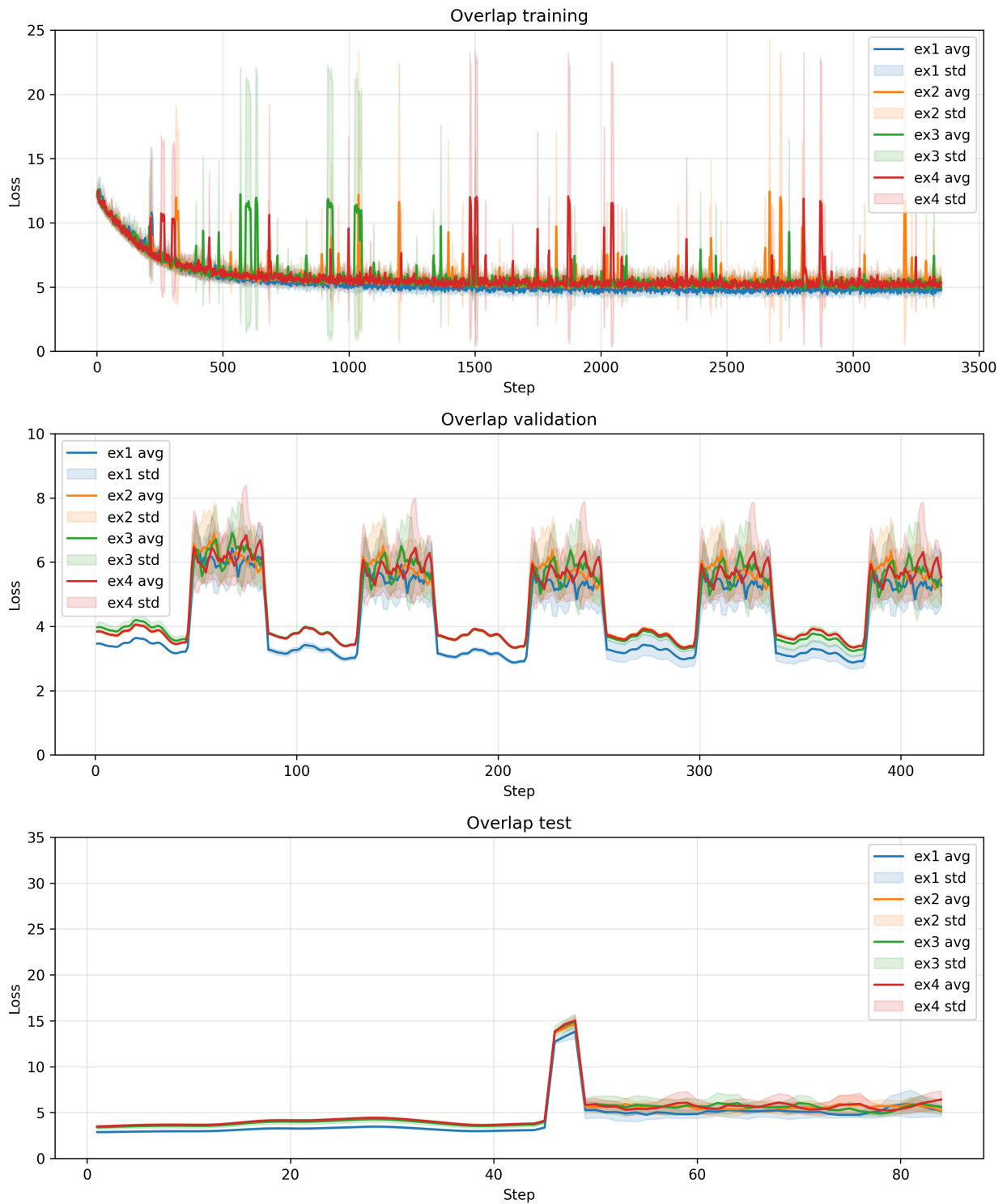


Figure 12: PINN_z overlap for tanh-modeller, med moving average med vindue størrelse på 3.

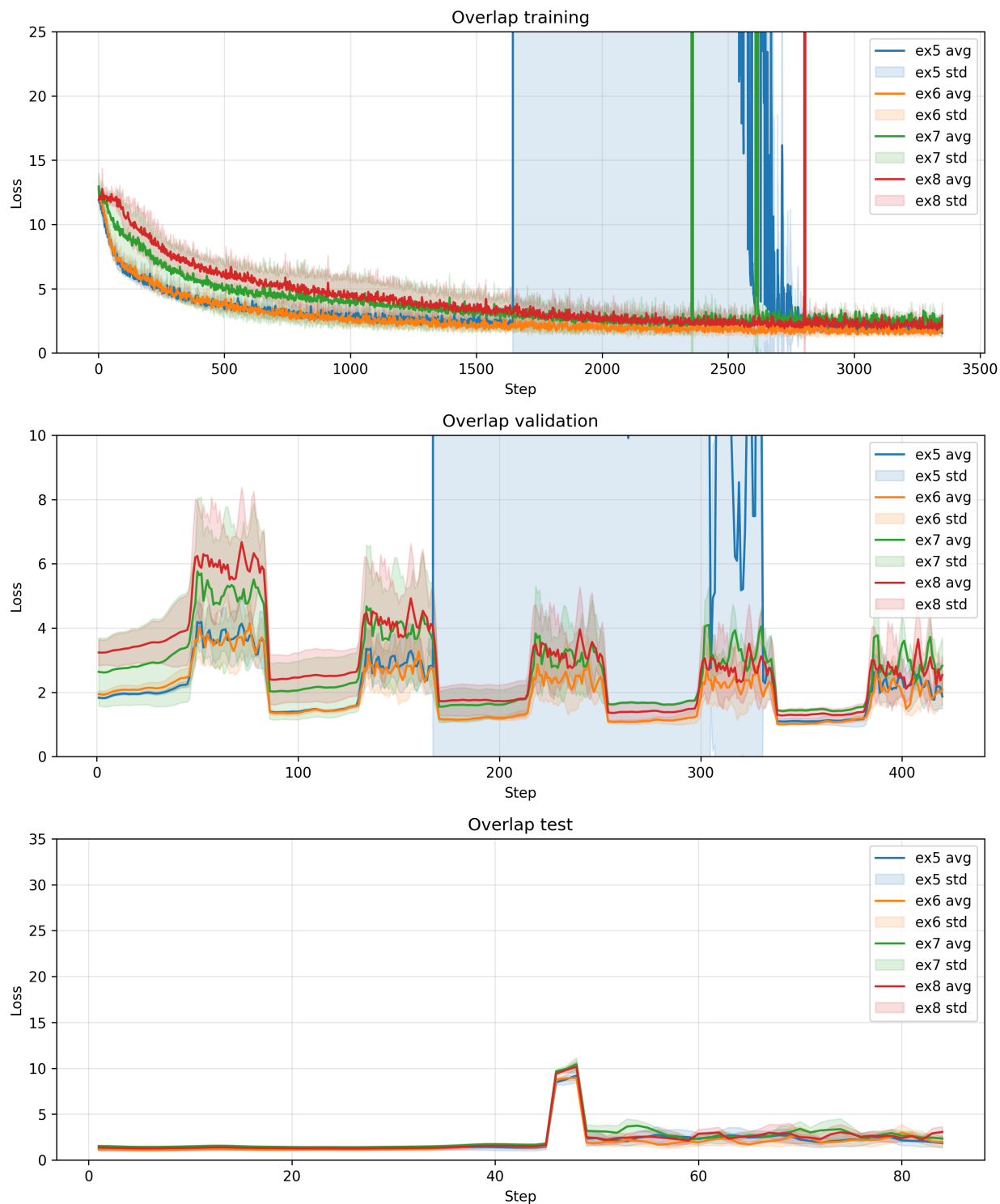


Figure 13: PINN_z overlap for SiLU-modeller, med moving average med vindue størrelse på 3.

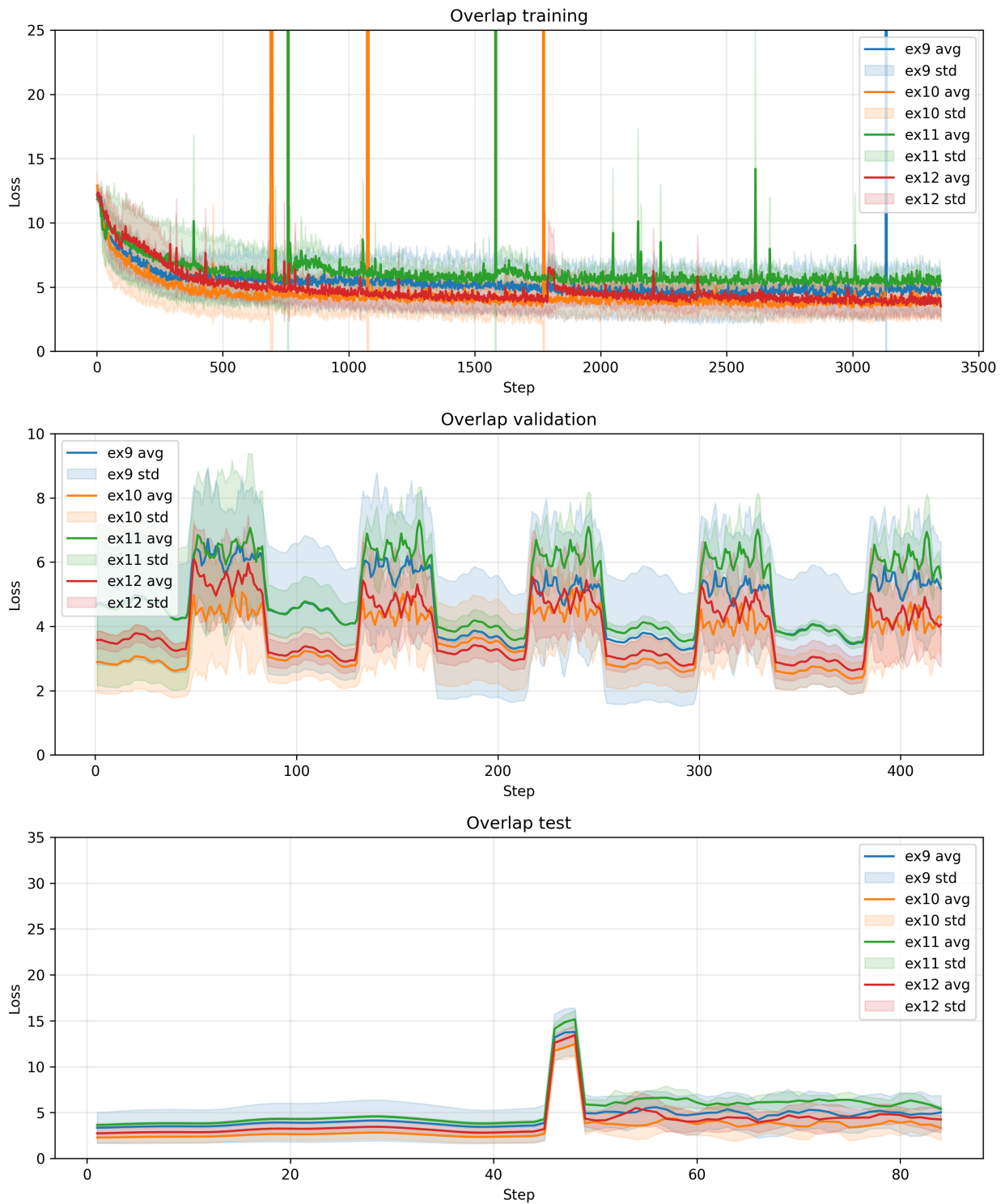


Figure 14: PINN_z overlap for blendede SiLU- og tanh-modeller, med moving average med vindue størrelse på 3.

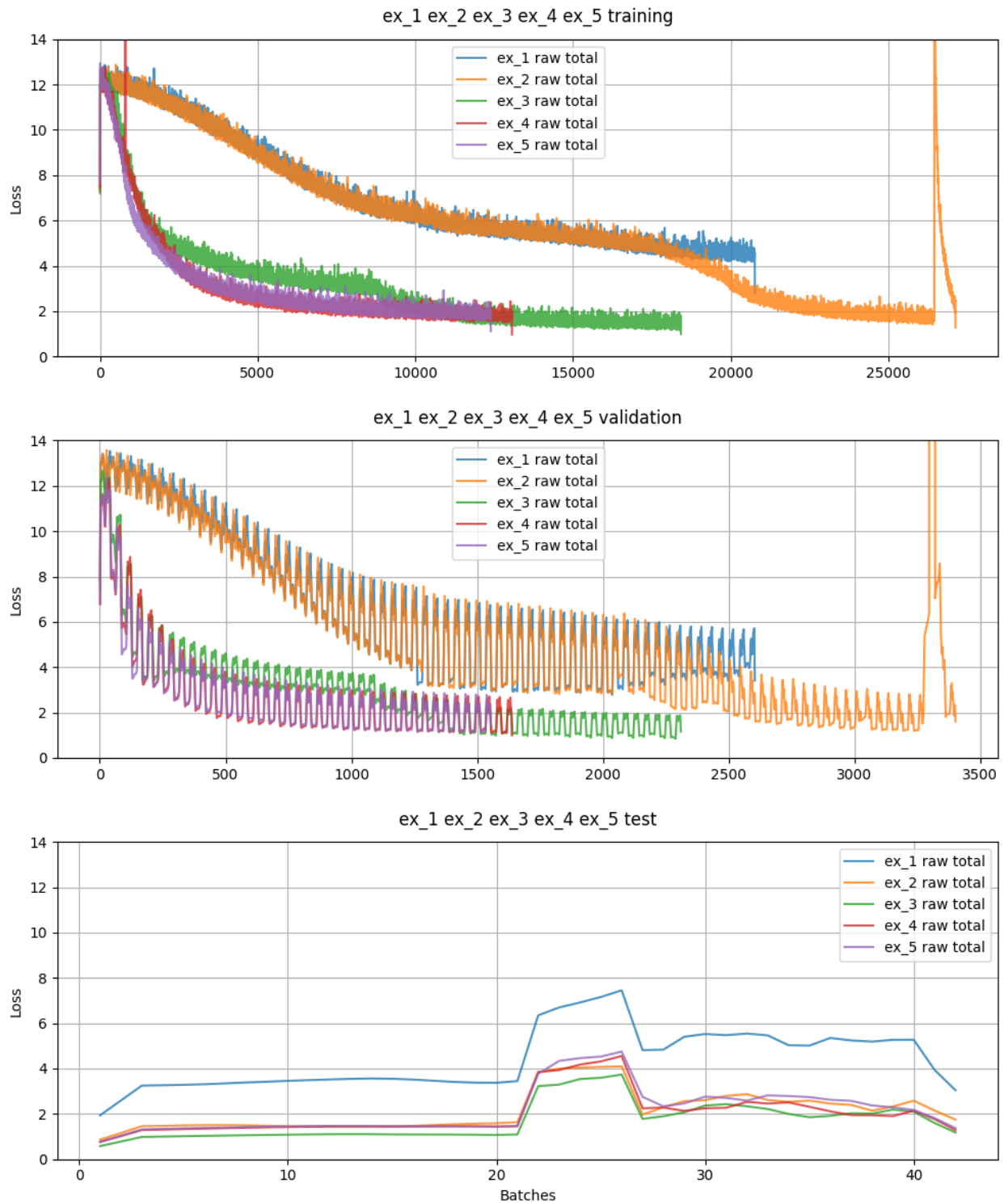


Figure 15: PINN_z modeller med 2 lag, med moving average med vindue størrelse på 5.

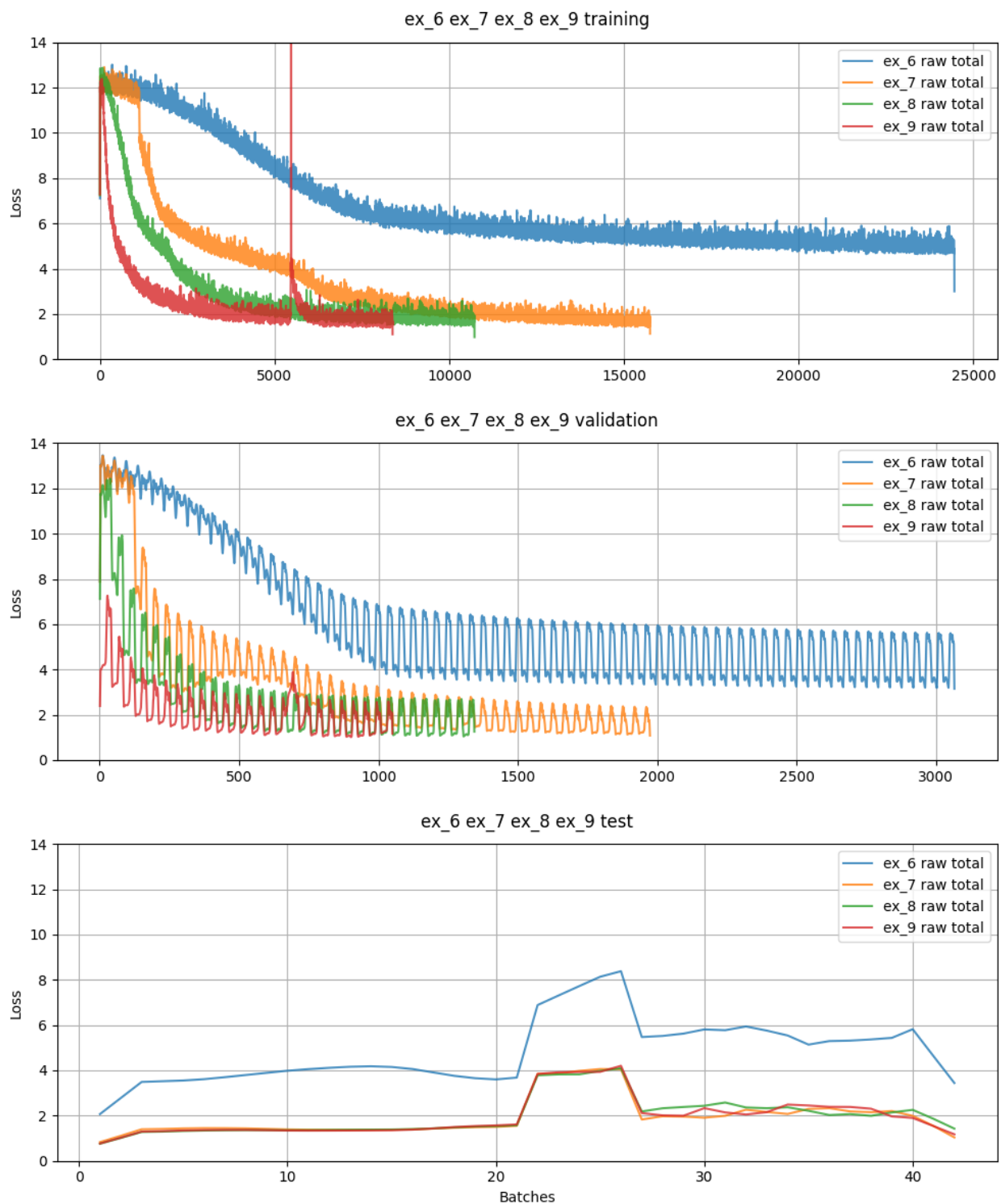


Figure 16: PINN_z modeller med 3 lag, med moving average med vindue størrelse på 5.

8 Appendix B

8.1 Anvendelse af Boltzmann ligning

Givet 2D rummet, med fordelingsfunktionen:

$$\begin{aligned}\beta &= (1, 2) \\ \mathbf{x} &= (x_1, x_2) \\ \mathbf{v} &= (v_1, v_2) \\ f_a(t, x_1, x_2, v_1, v_2) &= tx_1^2 + x_2v_1 + v_2^2\end{aligned}$$

Her ses bort fra kraftleddet ved at sætte $\frac{F_{a\beta}}{m_a} = \alpha$,

Ligningen er givet ved 13. For Einstein summation over β gælder:

$$\partial_{x_\beta}(v_\beta f_a) = \partial_{x_1}(v_1 f_a) + \partial_{x_2}(v_2 f_a).$$

Herved kan de afledte findes ved:

- Tidsafledte

$$\partial_t f_a = \partial_t(tx_1^2 + x_2v_1 + v_2^2) = x_1^2.$$

- Rumleddet

$$\partial_{x_\beta}(v_\beta f_a) = \partial_{x_1}(v_1 f_a) + \partial_{x_2}(v_2 f_a) = 2v_1tx_1 + v_1v_2$$

- Hastighedsleddet

$$\partial_{v_\beta}(\alpha f_a) = \partial_{v_1}(\alpha f_a) + \partial_{v_2}(\alpha f_a) = \alpha x_2 + 2\alpha v_2$$

- Samling med kollisionsleddet

$$x_1^2 + 2tx_1v_1 + v_1v_2 + \alpha x_2 + 2\alpha v_2 = C_a$$